

An Overview of Hierarchical Task Network Planning

Ilche Georgievski and Marco Aiello

*Distributed Systems Group
Johann Bernoulli Institute for Mathematics and Computer Science
University of Groningen*

e-mail: initial.surname@rug.nl

March, 2014

Abstract

Hierarchies are the most common structure used to understand the world better. In galaxies, for instance, multiple-star systems are organised in a hierarchical system. Then, governmental and company organisations are structured using a hierarchy, while the Internet, which is used on a daily basis, has a space of domain names arranged hierarchically. Since Artificial Intelligence (AI) planning portrays information about the world and reasons to solve some of world's problems, Hierarchical Task Network (HTN) planning has been introduced almost 40 years ago to represent and deal with hierarchies. Its requirement for rich domain knowledge to characterise the world enables HTN planning to be very useful, but also to perform well. However, the history of almost 40 years obfuscates the current understanding of HTN planning in terms of accomplishments, planning models, similarities and differences among hierarchical planners, and its current and objective image. On top of these issues, attention attracts the ability of hierarchical planning to truly cope with the requirements of applications from the real world. We propose a framework-based approach to remedy this situation. First, we provide a basis for defining different formal models of hierarchical planning, and define two models that comprise a large portion of HTN planners. Second, we provide a set of concepts that helps to interpret HTN planners from the aspect of their search space. Then, we analyse and compare the planners based on a variety of properties organised in five segments, namely domain authoring, expressiveness, competence, performance and applicability. Furthermore, we select Web service composition as a real-world and current application, and classify and compare the approaches that employ HTN planning to solve the problem of service composition. Finally, we conclude with our findings and present directions for future work.

Keywords: AI planning, Hierarchical Task Networks, Web Service Composition.

Contents

1	Introduction	2
1.1	Approach	3
1.2	Inclusion of HTN Planners and Studies	3
1.3	Web Service Composition	4
1.4	Running Example: Logistics Domain	4
1.5	Organisation	5
2	HTN Planning: Theory and Concepts	5
2.1	Theoretical Framework	5
2.2	Conceptual Framework	6
2.2.1	Task Decomposition	6
2.2.2	Constraints	7
2.2.3	Explicit Conditions	9
2.2.4	Search Space	10
2.3	Plan-Based HTN Planning	10
2.3.1	Review of Plan-Based HTN Planners	11
2.4	State-Based HTN Planning	16
2.4.1	Review of State-Based HTN Planners	17

3	Analysis of HTN Planning and HTN Planners	19
3.1	Analytical Framework	19
3.1.1	Domain Authoring	19
3.1.2	Expressiveness	20
3.1.3	Competence	20
3.1.4	Performance	21
3.1.5	Applicability	21
3.2	Outcome	22
3.2.1	Domain Authoring	22
3.2.2	Expressiveness	23
3.2.3	Competence	26
3.2.4	Performance	29
3.2.5	Applicability	30
4	HTN Planning for Web Service Composition	31
4.1	WSC Framework	31
4.2	WSC and HTN Planning	32
4.3	Review	33
4.4	Summary	36
5	Conclusions	36

1 Introduction

Hierarchical Task Network (HTN) planning is an Artificial Intelligence (AI) planning technique that breaks with the tradition of classical planning [1]. The basic idea behind this technique includes an initial state description, an initial task network as an objective to be achieved, and domain knowledge consisting of networks of primitive and compound tasks. A task network represents a hierarchy of tasks each of which can be executed, if the task is primitive, or decomposed into refined subtasks. The planning process starts by decomposing the initial task network and continues until all compound tasks are decomposed, that is, a solution is found. The solution is a plan which equals to a set of primitive tasks applicable to the initial world state.

Beside being a tradition breaker, HTN planning appears to be controversial as well. The controversy lies in its requirement for well-conceived and well-structured domain knowledge. Such knowledge is likely to contain rich information and guidance on how to solve a planning problem, thus encoding more of the solution than it was envisioned for classical planning techniques. This structured and rich knowledge gives a primary advantage to HTN planners in terms of speed and scalability when applied to real-world problems and compared to their counterparts in classical world.

The biggest contribution towards this kind of “popular” image of HTN planning has emerged after the proposal of the Simple Hierarchical Ordered Planner (SHOP) [2] and its successors. SHOP is an HTN-based planner that shows efficient performance even on complex problems, but at the expense of providing well-written and possibly algorithmic-like domain knowledge. Several situations may confirm our observation, but the most well-known is the disqualification of SHOP from the International Planning Competition in 2000 [3] with the motivation that the domain knowledge was not well written so that the planner produced plans that were not solutions to the competition problems [2]. Furthermore, the disqualification was followed by a dispute on whether providing such knowledge to a planner should be considered as cheating in the world of AI planning [4].

SHOP’s style of HTN planning was introduced by the end of 1990s, but HTN planning existed long before that. The initial idea of hierarchical planning was presented by the Nets of Action Hierarchies (NOAH) planner [5] in 1975. It was followed by a series of studies on practical implementations and theoretical contributions on HTN planning up until today. We believe that the fruitful ideas and scientific contribution of nearly 40 years must not be easily reduced to controversy and antagonism towards HTN planning. On the other hand, we are faced with a situation full of fuzziness in terms of difficulty to understand what kind of planning style other HTN planners perform, how it is achieved and implemented, what are the similarities and differences among these planners, and finally, what is their actual contribution to the creation of the overall and possibly objective image of HTN planning. The situation cannot be effortlessly clarified because the current literature reports little or nothing at all on any of these issues, especially in a consolidated form.

In addition to these issues, we observe the applicability of AI planning techniques as an ultimate goal of their development. We are especially interested in novel and real-world domains which may require reconsidering established

techniques. The growing trend on other than classical and synthetic domains leads to the need for algorithms and systems that reflect planning better and more closely to the real world. This perspective gives another view to the abilities of HTN planners (and HTN planning in general) to cope with various properties of an application in the real world.

We aim to consolidate and synthesise a number of existing studies on HTN planning in a manner that will clarify, categorise and analyse HTN planners, and allow us to make statements that are not merely based on contributions of a single HTN planner. We also hope to rectify the perception of HTN planning as being controversial and antagonistic in the AI planning community. Finally, we choose a non-traditional, dynamic and uncertain application domain to ascertain HTN planning with respect to various domain characteristics.

1.1 Approach

We take a framework-based approach to accomplish our objectives. We perceive a framework as an abstract and logical structure that we establish in need for support and guidance on the development of our study. Since we inspect HTN planning from four different perspectives, we provide a framework for each perspective. All four frameworks form the design depicted in Figure 1 to which we refer to as a *pie of frameworks*. The pie of frameworks serves as a central and unifying point of our study organisation and presentation flow. The first piece of the pie is a *theoretical framework* for HTN planning upon which we later build two models of HTN planning. The second slice is a *conceptual framework* for HTN planning that provides clarification of different concepts related to the search space, and context for interpretation of HTN planners. The next piece is an *analytical framework* for HTN planners that enables us to go deeper and beyond dry descriptions about HTN planners. The last piece might appear to taste differently than the other three, but it still has the flavour of HTN planning. The *application framework* concerns the application domain we choose to observe, and it helps us to analyse different studies in an organised and unified way, and possibly to identify points where HTN planning behaves as expected or can be further improved.

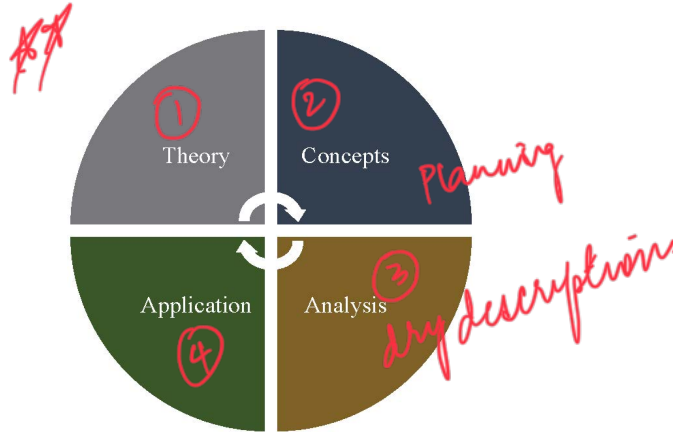


Figure 1: Pie of Frameworks

1.2 Inclusion of HTN Planners and Studies

We make use of two inclusion criteria for planners and studies. The *inclusion criterion of planners* relies on the inspection of existing literature for suggestions on HTN planners that have risen to some degree of prominence. For example, we accept the list of “best-known domain-independent HTN planning systems” as provided in [1]. In addition to those five suggested planners, we include two more. The complete list of HTN planners participants in our study is the following one:¹

- Nets Of Action Hierarchies (NOAH), the first HTN planner emerged in mid-1970s [6, 5],
- Nonlin that appeared one year later [7, 8],
- System for Interactive Planning and Execution (SIPE) and its successor SIPE-2 introduced in 1984 and 1990, respectively [9],
- Open Planning Architecture (O-Plan) and its successor O-Plan2 presented in 1984 and 1989, respectively [10, 11],

¹Later in the study, we refer only to the most recent version of each planner.

- Universal Method Composition Planner (UMCP) introduced in 1994 [12],
- Simple Hierarchical Ordered Planner (SHOP) and its successor SHOP2 that appeared in 1999 and 2003, respectively [2, 13], and
- SIADEx that emerged in 2005 [14].

The *inclusion criterion of studies* relies on the theoretical contribution of a study with respect to HTN planning in general, and theoretical and practical issues of each chosen planner separately. The criterion is based on the coverage a study gives, which may include information that ranges from a general discussion of techniques and approaches, peculiar matters, such as task interactions and condition types, relevant to our conceptual framework, to properties, such as domain authoring, expressiveness and competence, that may be a part of the analytical framework. Finally, we include an extensive number of studies that employ HTN planning for the purpose of our application domain, that is, Web service composition.

1.3 Web Service Composition

We choose the domain of Web services as a non-traditional and real-world application [15]. *Web services* are software components that implement specific business logic, and are distributed over a network, typically the Web, to be used as Web resources for machine-to-machine interaction. For instance, travel agencies may provide a number of Web services, such as booking a flight ticket, reserving a hotel, renting a car, or organising sightseeing. The interaction is usually initiated by a client request which has to be satisfied by the functionalities that Web services offer. However, in cases when no single service can accomplish the request, a combination of several Web services might give a value-added functionality, and provide a way to request satisfaction. For example, a service to arrange a complete trip to some tourist destination might be of an exceptional use to the commercial travel agencies, and thus, it will not be offered as a Web service. *Web service composition* (WSC), especially when accomplished automatically, appears to be in-line with the objective of AI planning [16, 17, 18]. That is, planning operators correspond to functionalities of Web services, while the goal, in the simplest example, is aggregated from the user request. In addition, the environment of Web services already shows some propensity to composite or hierarchical representation. For example, a composite service could describe a reservation of hotel by searching for a hotel, registering to the hotel, logging to the hotel and, finally, the actual booking of the hotel. Among AI planning techniques, HTN planning is well-suitable for domains in which some hierarchical representation is desirable or known in advance, domains that encourage complex and composed constructs, and domains of large size. These indicators suggest ideal conceptual matchmaking between HTN planning and Web service composition, but are also a computational challenge. Thus, continuing with our example, if the user objective is not only to reserve a hotel, but to arrange a complete trip, which includes also booking a flight, renting a car, and sightseeing, then, definitely, the complexity of services and their composition becomes an interesting and challenging task.

The environment of Web services offers more exciting challenges that make the effective selection and composition of services far from being plain and straightforward planning processes. In particular, Web services exist in a *dynamic* environment in which the availability of services is not guaranteed. This behaviour reflects the availability of information which, on the other hand, is assumed by planners to be complete and obtainable before the planning process is initiated. Furthermore, the environment of Web services favours techniques that are able to deal with *uncertainty* in terms of 1) incomplete information about the initial state; 2) uncertainty over the many possibilities for completion of missing information by invoking some sensing services at planning and/or execution time; 3) non-determinism caused by failed invocations of Web services (e.g., renting a car is not viable at the moment of invocation), a service not responding at all, a service yielding undesired outcome (e.g., booking a flight provides only business-class tickets); 4) services that show unexpected behaviour (e.g., Byzantine failure). Moreover, *complex goals* possibly in the form of a workflow or conditioned with some organisational regulations or augmented with user preferences are the norm rather than exception. Finally, the *high cardinality* of the set of Web services available on the Web implies a large space to be searched by a planner.

1.4 Running Example: Logistics Domain

In order to illustrate some of the concepts and definitions discussed throughout the paper, we take as an example the logistics planning domain. Figure 2 illustrates a scenario using the domain. Suppose a setting in the logistic world that contains several locations, such as 11, 12 and 13, which can be adjacent (the solid line) and in a same city (the dash-dotted circle) or in different cities (the dashed line), a box *b* that should be delivered, and a truck *t* and a plane *p*, which are delivery vehicles for a box transported in same-city locations or different-city locations, respectively. The goal is to deliver a box from an initial location to a desired destination. For example, a goal could say to deliver

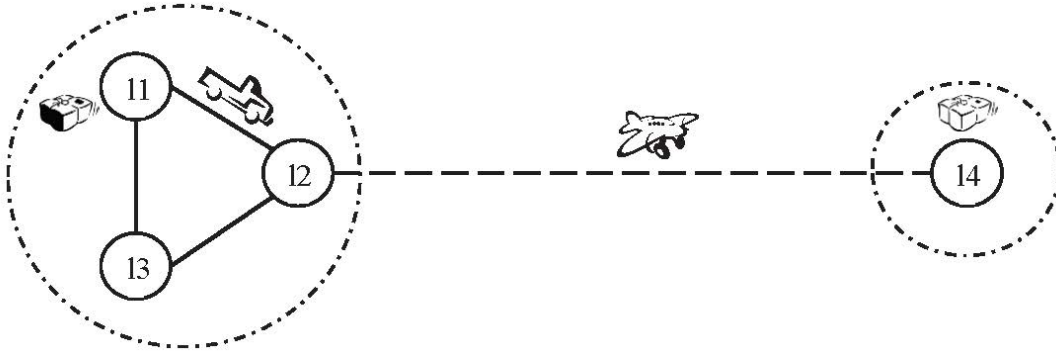


Figure 2: An Example in the Logistics Domain

a box from 11 to 14. The box can be transported by a truck by loading the box into the truck at 11 (denoted as `load-truck(t,b,11)`), driving the truck to 12 (denoted as `drive(t,11,12)`), and unloading the truck at 12 (denoted as `unload-truck(t,b,12)`). Finally, loading the box into a plane, flying the plane to 14, and unloading the plane at 14 are denoted analogously to their counterparts.

1.5 Organisation

The remainder of the paper is organised as follows. Section 2 describes the theoretical and conceptual frameworks. Based on the ideas presented in these frameworks, two models of HTN planning are proposed and formalised. The planners corresponding to an HTN model are reviewed with respect to the conceptual framework. Section 3 provides details on each HTN planner separately. Section 4 goes deeper into the application area of Web service composition accomplished by HTN planning. Finally, Section 5 concludes the paper with some considerations and directions for future work.

2 HTN Planning: Theory and Concepts

HTN planning has been formalised in several studies, such as [19, 2, 1, 20]. These formalisations include similar definitions of HTN terms, appropriate to the needs for their model of HTN planning. Based on these existing theories, we describe the first piece of our framework pie, that is, the theoretical framework for HTN planning. In this framework, we keep the definitions of HTN terms high level. Further in the paper, we provide specific definitions of the terms that are characteristic for the model of HTN planning being analysed. The purpose of the theoretical framework is twofold. Firstly, it provides basic understanding of HTN planning. Secondly, it determines and defines the focus of categorisation of HTN planning that we propose.

2.1 Theoretical Framework

The theoretical framework is composed of a planning language, tasks, operators, task networks, methods, planning problem and solution. The *HTN planning language* is a first-order language that contains several mutually disjoint sets of symbols. As usual, a *predicate*, which evaluates to true or false, consists of a predicate symbol $p \in P$, where P is a finite set of predicate symbols, and a list of terms $\tau_1, \dots, \tau_k$. A *term* is either a constant symbol $c \in C$, where C is a finite set of constant symbols, or a variable symbol $v \in V$, where V is an infinite set of variable symbols. We denote the set of predicates as Q . A predicate is *ground* if its terms contain no variable symbols. A *state* $s \in 2^Q$ is a set of ground predicates in which the *closed-world assumption* is adopted, that is, all and only the predicates that are true are specified in the state. We define a *primitive task* as an expression $t_p(\tau)$, where $t_p \in T_p$ and T_p is a finite set of primitive-task symbols, and $\tau = \tau_1, \dots, \tau_k$ are terms. A primitive task is represented by a planning operator.

Definition 1 (Operator). An *operator* o is a triple $(p(o), pre(o), eff(o))$, where $p(o)$ is a primitive task, and $pre(o) \in 2^Q$ and $eff(o) \in 2^Q$ are preconditions and effects, respectively. The subsets $pre^+(o)$ and $pre^-(o)$ denote positive and negative preconditions of o , respectively.

A transition from one state to another is accomplished by an instance of an operator whose precondition is a logical consequence of the current state. An operator o is *applicable* in state s , if $pre^+(o) \subseteq s$ and $pre^-(o) \cap s = \emptyset$. Applying o to s results in the state $s[o] = (s \setminus eff^-(o)) \cup eff^+(o)$, where $eff^-(o)$ and $eff^+(o)$ are negative and positive effect of o , respectively. Notations $s[o] = s'$ and $s \xrightarrow{o} s'$ are equivalent and we use them interchangeably.

A *compound task* is an expression $t_c(\tau)$, where $t_c \in T_c$ and T_c is a finite set of compound-task symbols, and $\tau = \tau_1, \dots, \tau_k$ are terms. We refer to the union of the sets of primitive-task and compound-task symbols as a set of task names T_n . The following two definitions are further complemented for the respective model of HTN planning.

Definition 2 (Task network). A *task network* tn is a pair (T, ψ) , where T is a finite set of tasks, and ψ is a set of constraints.

Constraints in ψ specify restrictions over T that must be satisfied during the planning process and by the solution. We refer to a task network over T_p as a *primitive task network*. The set of all task networks over T_n is denoted as TN .

Definition 3 (Method). A *method* m is a pair $(c(m), tn(m))$, where $c(m)$ is a compound task, and $tn(m)$ is a task network.

Definition 4 (Planning problem). A *planning problem* $\mathcal{P}$ is a tuple $(Q, T_p, T_c, O, M, tn_0, s_0)$, where

- Q is a finite set of predicates
- T_p is a finite set of primitive task symbols
- T_c is a finite set of compound task symbols
- $O \subseteq T_p \times 2^Q \times 2^Q$ is a finite set of operators
- $M \subseteq T_c \times TN$ is a finite set of methods
- tn_0 is the initial task network
- s_0 is the initial state.

An operator sequence $o_1, \dots, o_n$ is *executable* in s , if there are states $s_0, \dots, s_n$ such that $s_0 = s$ and o_i is applicable in s_{i-1} and $s_{i-1}[o_i] = s_i$ for all $a \leq i \leq n$. Given a problem $\mathcal{P}$, a *solution* to $\mathcal{P}$ is an operator sequence executable in s_0 by decomposing tn_0 . The way of producing such sequence is defined in the following sections.

2.2 Conceptual Framework

Literature reports vague information on HTN planning (and planners) especially in the early stages of hierarchical planning. First, it is difficult to understand the ideas and concepts used and how they are adapted for the purpose of HTN planning. The situation later improved with the evolution of HTN planners and the attempts at formalisation. Then, different models of HTN planning could be found at some point of the evolution, and at first glance, the model distinction seems not that obvious and comprehensible.

The motivation for the second slice of our framework pie lies exactly in these issues. We clarify them by designing and describing a conceptual framework shown in Figure 3. This framework is less formal (compared with the theoretical framework) and based on specific concepts derived from empirical observation. We start by providing basic and general enough descriptions of concepts that characterise different HTN planners and cover most of their important features. The concepts are placed within a logical and sequential design as much as possible. In our framework, the key concept is the search space to which other concepts are related and interconnected in various ways. The purpose of the conceptual framework is manifold. Firstly, it clarifies concepts and proposes relationships among them. Secondly, it provides context for interpreting the findings presented in the paper, and helps in explaining observations. Finally, the view of key concepts enables us to categorise HTN planning based on the space the search is performed in.

2.2.1 Task Decomposition

Given some task network, a *task decomposition* chooses a task from the task network and, if it is primitive and suitable for the current state, the task decomposition applies it to the state. Otherwise, all the methods are analysed that contain the chosen task as a part of their definition. Assuming that a set of methods is found, a non-deterministic choice of a method is made, and the task is replaced with the task network associated with the chosen method. Finally, the newly composed task network is checked against any constraint-related violation and modified, if necessary.

Task decompositions can be divided into three styles based on the representation of task networks in terms of task ordering, and the way of forming new task networks during decomposition. The first one is *totally ordered task decomposition* (TOTD). It follows the assumption of total order on task networks so as when a task is decomposed, the new task network is created in such a way that newly added tasks are totally ordered among each other and with respect to the tasks of the existing task network. Sometimes we refer to the HTN planning that uses this style as

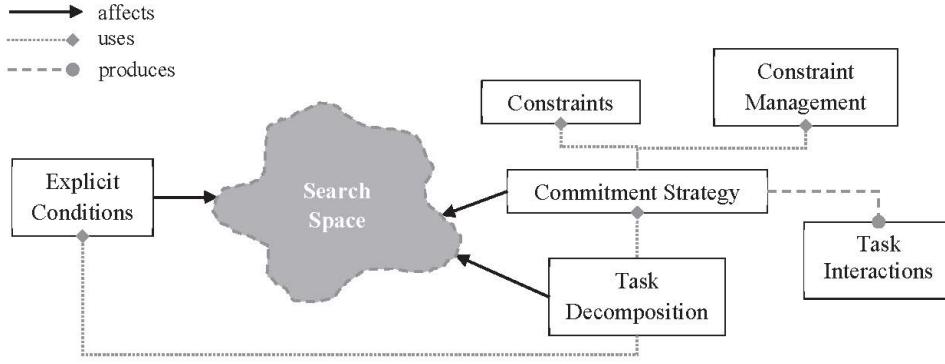


Figure 3: Conceptual Framework

totally ordered HTN planning. The second style is *unordered task decomposition* (UTD) that relaxes the requirement of totally ordered task networks. That is, tasks can be totally ordered or unordered with respect to each other (but no tasks in parallel are allowed). When a task is decomposed, new task networks are created in such a way that newly added tasks are interleaved with the tasks of the existing task network until all permissible permutations are exhausted. Here as well, we refer to the HTN planning that embodies this style as unordered HTN planning. The last style is *partially ordered task decomposition* (POTD) that allows the existence of a partial order on tasks. When a task is decomposed, the tasks in the newly created network can be ordered in parallel whenever possible (with respect to the constraints). The HTN planning that uses this style is referenced as partially ordered HTN planning.

2.2.2 Constraints

Definition 2 suggests that constraints are found in a task network, but constraints can be also added during the planning process in order to resolve inconsistencies. HTN planners deal with several types of constraints, and most of them can be interpreted as in [21]. Briefly, there are three interpretations. First, we meet a constraint that implies commitments about partial descriptions of state objects. Another type of constraint refines variable bindings if a certain variable binding does not satisfy some condition. Last, there is a constraint that expresses the relations between variables in different parts of a task network.

Commitment Strategy As with most of the other AI planners, HTN planners also need to make two decisions on constraints. The first one is on constraints for bindings variables, while the second decision is on constraints for ordering tasks in a task network. HTN planners use mainly two strategies for when and how to make these decisions. The first strategy manages constraints in compliance with the *least-commitment strategy* so that task ordering and variable bindings are deferred until a decision is forced [22, 23]. The second strategy handles constraints according to the *early-commitment strategy* so that variables are bound and operators in the plan are totally ordered at each step of the planning process. Planners employing this strategy greatly benefit from the possibility of adopting forward chaining where chaining of actions is achieved by imposing a total order on (some) plan actions. The total ordering ensures that neither the current action to be added to the plan can interfere with some earlier action’s preconditions or effects, nor a later action can interfere with current action’s preconditions or effects.

Task Interaction Depending on the commitment strategy chosen, and especially in the case of the least-commitment strategy, an inevitable consequence is the interaction among tasks in a given task network. Generally, an *interaction* is a connection between two tasks (or parts) of a task network in which these tasks (or parts) have some effect on each other. Based on this effect, we divide interactions into two categories. The first category, called *harmful interactions* (also threats or flaws), introduces conflicts among different parts of a task network that threaten its correctness. HTN planners consider harmful interactions individually, and provide rather intuitive descriptions. In the following list, we provide a general description of harmful interactions found in HTN planners.

- *Deleted-condition interaction* - appears when a primitive task in one part of a task network deletes an expression that is a precondition to a primitive task in another part of that task network. For example, consider the following situation from our running-example domain. The state contains three possible locations l1, l2 and l3, one truck t1, and one box b to be transported. The idea is to move b1 from l1 to l2 and to move t1 from l2 to l3. Then, the task network depicted in Figure 4a has a deleted-condition interaction. The conflict arises

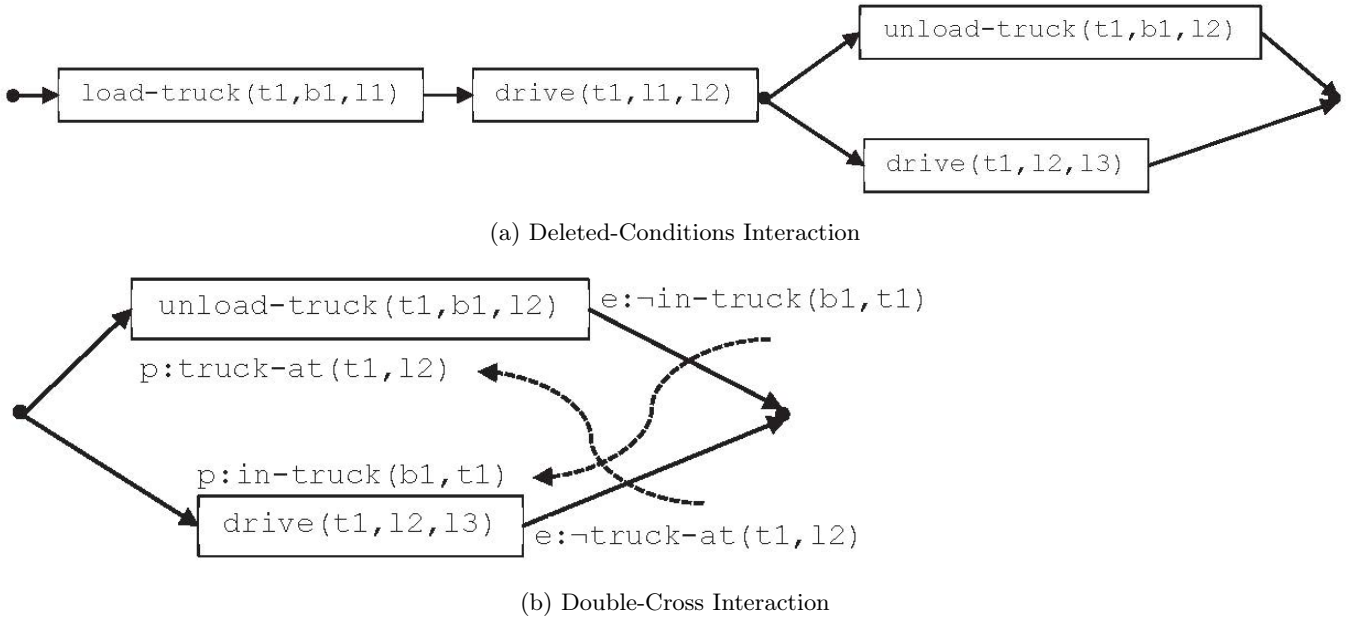


Figure 4: Examples of Deleted-Condition and Double-Cross Interactions

when the action `drive(t1,l2,l3)` deletes the fact `truck-at(t1,l2)`, which is added by `drive(t1,l1,l2)` and is a precondition of `unload-truck(t1,b1,l2)`.

- *Double-cross interaction* - appears when an effect of each of two conjunctive primitive tasks deletes a precondition for the other. That is, an effect of the first task deletes a precondition of the second primitive task, and an effect of the second task deletes a precondition of the first task. For example, consider the situation shown in Figure 4b, where a truck `t1` is at location `l2` and a box `b1` is loaded into the `t1`. The idea is to unload the truck at `l2` and move the truck to `l3`, and in addition, for the purpose of illustration, consider an expanded description of the `drive(t,b,l)` operator that requires a box to be in the truck while moving. Then, tasks are in a double-cross interaction, that is, `unload-truck(t1,b1,l2)` deletes `in-truck(b1,t1)`, which is a precondition for `drive(t1,l2,l3)` in order to move to `l3`, and `drive(t1,l2,l3)` deletes `truck-at(t1,l2)`, which is a precondition for the execution of `unload-truck(t1,b1,l2)`.
- *Resource interaction* - appears in two situations, and it is subdivided accordingly. A *resource-resource interaction* is similar to the deleted-condition interaction, while a *resource-argument interaction* occurs when a resource in one part of a task network is used as an argument in another part of that task network.

The second category, called *helpful interactions*, refers to situations when one part of a task network can make use of information associated with another part in the same task network. The detection of these interactions implies the possibility for a planner to generate better-quality task networks and solutions. In fact, some tasks can be merged together, which eliminates task redundancy and potentially optimises the cost of the solution [24]. The following list contains general descriptions of helpful interactions.

- *Placeholder replacement* - appears when a real value already exists for a particular formal object. HTN planners allow tasks with variables to be inserted into a task network. If there is no specific value to be chosen for a particular variable choice, a so-called formal object is created to bind the variable [6]. The formal object is simply a placeholder for some entity unspecified at that point.
- *Phantomisation* - appears when some goal is already true at the point in a task network where it occurs. In the descriptions of some HTN planners, the term ‘goal’ is interchangeably used with the term ‘precondition’. In fact, if some task precondition is not satisfied, it is inserted as a goal to be achieved.
- *Disjunct optimisation* - appears in disjunctive goals when one disjunctive goal is “superior to the others by the nature of its interaction” with the other tasks in a task network [6].

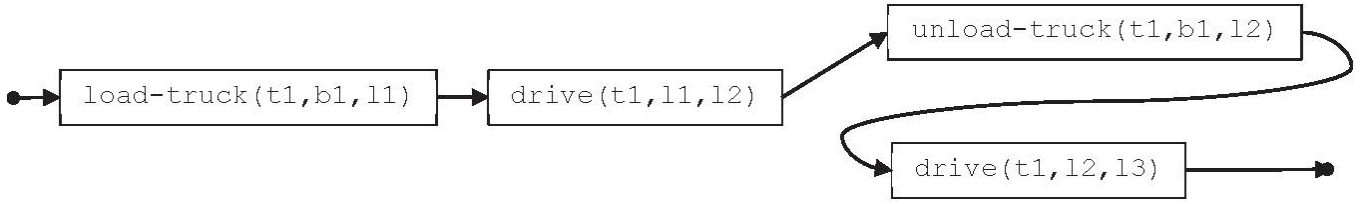


Figure 5: A Conflict-Free Portion of a Task Network as a solution to the interaction in Figure 4a

Constraint Management Task interactions can be solved by posting various types of constraints onto a task network. This *constraint posting* is also known as conflict resolution [25] or critics [7, 26]. Similarly as for the representation, HTN planners do not provide a general approach for handling interactions, thus each of the above interactions has its own resolution method. However, those methods are based on well-known operations on constraints generally described elsewhere, e.g., [21]. We briefly describe some operations in the context of HTN planning.

The most basic operation is *constraint satisfaction* which happens when an HTN planner searches for a variable binding that satisfies the given constraints, and guarantees the consistency of, for example, a set of ordering constraints over a task network. *Constraint propagation* enables adding or retracting constraints to and from a task network. Variable constraints in one part of a task network can be propagated based on variable constraints in another part of that task network. With respect to ordering constraints, propagation is used when the linking process is performed. When some task interferes with another task, the *linking process* records a causal link, that is, a three-element structure of two pointers to tasks t_e and t_p , and a predicate q which is both an effect of t_e and a precondition of t_p . For example, the deleted-condition interaction from Figure 4a is resolved by adding a constraint to execute action `unload-truck(t1,b1,l2)` before `drive(t1,l2,l3)`, which results in a conflict-free task network as shown in Figure 5. The solution to the phantomisation interaction is practically a linking process. Phantomisation of a task t with an effect e is accomplished by treating e as achieved, and finding an existing task t' in the task network that achieves the same effect e . If task t' is found, a constraint (t', e, t) is added to the task network to record the causal relation.

The last operation is different in that it does not happen during the planning process. *Constraint formulation* can be taken into account when modelling HTN domain knowledge, especially when the domain author is aware in advance of some possible impasse situations. By posting constraints as control information into the domain knowledge, the planner can gain on efficiency by refining the search space [27, 28]. In some HTN planners, the phantomisation of a task is achieved by an explicit encoding in the domain knowledge. The planners handle the phantomisation of a rather recursive task by taking into account an alternative method which explicitly encodes a ‘do-nothing’ operation. Therefore, one could say that the planners lack the ability to infer such situations. This issue is addressed in [29], where a support for reasoning about the conditions that are already achieved without the need of explicit specification into the domain knowledge is provided.

2.2.3 Explicit Conditions

HTN planners depend on the quality of the domain knowledge to restrict and guide the search. The domain author is the one who has the responsibility of giving the information about the guidance in the search space. One way to represent such information is by using explicit conditions. We provide a general description of conditions found in HTN planners.

- *Supervised condition* - accomplished within a compound task. The condition may be satisfied either by an intentional insertion of a relevant effect earlier in the task network, or by an explicit introduction of a primitive task that will achieve the required effect. Generally, only this condition should allow further decompositions to be made and, since it may be included for the achievement of the condition invocation of another task, this condition corresponds to preconditions in STRIPS-like planning systems.²
- *External condition* - must be accomplished at the required task, but under the assumption that it is satisfied by some other task from the task network. An external condition can be seen as a sequencing constraint.
- *Filter condition* - decides on task relevance to a particular situation. In the case of method relevance to a certain task decomposition, this condition reduces the branching factor by eliminating inappropriate methods.
- *Query condition* - accomplishes queries about variable bindings or restrictions at some required point in a task network.

²With the “STRIPS-like” term, we refer to planning languages that use representations similar to the STRIPS one [30], that is, an operator is composed of preconditions, a delete list and an add list.

- *Compute condition* - requires satisfaction by information coming only from external systems, such as a database.
- *Achieve condition* - allows expressing goals that can be achieved by any means available to a planner.

Note on Filter Conditions According to Erol et al. [12], filter conditions can be used to prune the search space. On the contrary, Kambhampati discusses that filter conditions can be seen as selection heuristics rather than pruning mechanisms [31]. In particular, preferring task networks with already established filter conditions is not the same as pruning. Kambhampati argues that filter conditions should be seen as an integral part of the fashion of allowing the domain author greater control over the types of generated solutions, that is, filter conditions enable the domain author to disallow certain types of solutions. In Kambhampati’s view, the loss of completeness using filter conditions, a point made by [32], is a ramification of the erroneous interpretation and implementation of filter conditions.

2.2.4 Search Space

So far we described concepts that affect the structure of the space to be searched. Next, we intuitively describe two structures of search spaces created by HTN planners. The first type of space consists of task networks and task decompositions as evolutions from one task network to another. Given some planning problem, at the beginning of the search, a task decomposition is imposed on the initial task network, and the process continues by repeatedly decomposing tasks from a newly created task network until a primitive task network is produced. A linearisation of this primitive task network executable in the initial state represents a solution to the planning problem.

The second type of search space is a subset of the state space. The subset consists of explicitly described states restricted by task decompositions. As in the classical state space, the search begins in the initial state with an empty plan, but instead of searching for a state that will satisfy the goal, the search is for a state that will accomplish the initial task network. In particular, if a task from the task network is compound, a task decomposition is performed and the search continues on the next decomposition level, but in the same state. If the task is primitive, it is executed and the search continues into a successor state. This task is then added to the plan. When there are no more tasks in the task network to be decomposed, the search is finished. The solution to the planning problem is the plan containing a sequence of totally ordered actions.

Categorisation of HTN planners In the first type of search space, the initial task network is reduced to a primitive task network that constitutes a solution to the planning problem. At each point in the space, the task network can be seen as a partially specified plan until the search reaches the point where the task network is primitive and represents a solution plan. Thus, we employ the term *plan space* to refer to this type of search space. We refer to HTN planners that search in this plan space as *plan-based HTN planners*, and to the model of HTN planning as *plan-based HTN planning*. For the obvious reasons, we employ the term *state space* to refer to the second type of search space. Thus, we refer to HTN planners searching in this space as *state-based HTN planners*, and to the model of HTN planning as *state-based HTN planning*.

2.3 Plan-Based HTN Planning

We draw the formalism of plan-based HTN planning upon the work of Geier and Pascal [20]. With respect to the Definition 2 of the theoretical framework, we complement a task network as follows.

Definition 5 (Task network). A *task network* tn is a triple (T, φ, ψ) , where

- T is a finite and non-empty set of tasks
- $\varphi : T \rightarrow T_n$ labels a task with a task name
- ψ is a formula composed by conjunction, disjunction or negation of the following sets of constraints:
 - $\prec \subseteq T \times T$ is a strict partial order on T (irreflexive, transitive, asymmetric)
 - $\mapsto \subseteq V \times V \cup V \times C$ is a restriction on bindings of task network variables
 - $\vdash_{\prec} \subseteq T \times Q \cup Q \times T \cup T \times Q \times T$ is a partial order on tasks and state predicates.

Since some task name can occur many times in one task network, task labelling enables identifying uniquely many occurrences of that task name. For example, $tn = (\{t_1, t_2, t_3\}, \{(t_1, t'), (t_2, t''), (t_3, t')\}, \emptyset)$ denotes that the task network consists of two tasks associated with task name t' and one task associated with t'' .

A task network $tn = (T, \varphi, \psi)$ is *isomorphic* to $tn' = (T', \varphi', \psi')$, denoted as $tn \equiv tn'$, if and only if there exists a bijection $\beta : T \rightarrow T'$, such that

- for all $t, t' \in T$ it holds $(t, t') \in \prec$ if and only if $(\beta(t), \beta(t')) \in \prec'$
- for all $v_1, v_2 \in V$ and $c \in C$ it holds $(v_1, v_2) \in \mapsto$ or $(v_1, c) \in \mapsto$ if and only if there exist $v'_1, v'_2 \in V$ and $c' \in C$ such that $v_1 = v'_1$, $v_2 = v'_2$ and $(v'_1, v'_2) \in \mapsto'$ or $v_1 = v'_1$, $c = c'$ and $(v'_1, c) \in \mapsto'$
- for all $t, t' \in T$ and $p \in Q$ it holds $(t, p) \in \vdash_{\prec}$ or $(p, t) \in \vdash_{\prec}$ or $(t, p, t') \in \vdash_{\prec}$ if and only if $(\beta(t), p) \in \vdash'_{\prec}$ or $(p, \beta(t)) \in \vdash'_{\prec}$ or $(\beta(t), p, \beta(t')) \in \vdash'_{\prec}$

and $\varphi(t) = \varphi'(\beta(t))$.

Definition 6 (Decomposition). Let m be a method and $tn_c = (T_c, \varphi_c, \psi_c)$ be a task network. Method m *decomposes* tn_c into a new task network tn_n by replacing task t , denoted as $tn_c \xrightarrow[t, m]{t, m} tn_n$, if and only if $t \in T_c$, $\varphi_c(t) = c(m)$, and there exists a task network $tn' = (T', \varphi', \psi')$ such that $tn' \equiv tn(m)$ and $T' \cap T \neq \emptyset$, and

$$\begin{aligned} tn_n := & ((T_c \setminus \{t\}) \cup T', \varphi_c \cup \varphi', \psi_c \cup \psi' \cup \psi_D) \text{ where} \\ \psi_D := & \{(t_1, t_2) \in T_c \times T' \mid (t_1, t) \in \prec_c\} \cup \{(t_1, t_2) \in T' \times T_c \mid (t, t_2) \in \prec_c\} \cup \\ & \{(p, t_1) \in Q \times T' \mid (p, t) \in \vdash_{\prec_c}\} \cup \{(t_1, p) \in T' \times Q \mid (t, p) \in \vdash_{\prec_c}\} \cup \\ & \{(t_1, p, t_2) \in T' \times Q \times T' \mid (t, p, t_2) \in \vdash_{\prec_c}\} \end{aligned}$$

Given a planning problem $\mathcal{P}$, $tn_c \rightarrow_D^* tn_n$ indicates that tn_n results from tn_c by an arbitrary number of decompositions using methods from M .

Definition 7 (Executable Task Network). Given a planning problem $\mathcal{P}$, $tn = (T, \varphi, \psi)$ is *executable* in state s , if and only if it is primitive and there exists linearisation of its tasks $t_1, \dots, t_n$ that is compatible with ψ and the corresponding sequence of operators $\varphi(t_1), \dots, \varphi(t_n)$ is executable in s .

Definition 8 (Solution). A task network tn_s is a solution to a planning problem $\mathcal{P}$, if and only if tn_s is executable in s_0 , and $tn_0 \rightarrow_D^* tn_s$ for tn_s being a solution to $\mathcal{P}$.

Our definition of the plan space is similar to the definition of the decomposition problem space in [33]. Intuitively, a problem space is a directed graph in which task networks are vertices, and a decomposition of one task network into another task network by some method is an outgoing edge, under the condition that the initial task network belongs to the graph. More precise definition follows.

Definition 9 (Plan space). Given a plan-based HTN planning problem $\mathcal{P}$, a *plan space* PG is a directed graph (V, E) if and only if $tn_0 \in V$, and for each $tn \rightarrow_D tn'$: $tn, tn' \in V$ and $(tn, tn') \in E$.

An example of a HTN plan space is illustrated in Figure 6.

2.3.1 Review of Plan-Based HTN Planners

Task Decomposition

Most plan-based HTN planners perform task decomposition in a slightly different way than the process described in Section 2.2.1. The main reason lies in the approach that these planners use to distinct and represent tasks. In fact, with the exception of UMCP, the rest of the planners support only a single structure to encode both primitive and compound tasks. Although it is not always clear what is the purpose of a respective structure or how task decomposition is accomplished, we try to describe the approach that each planner takes to represent a task, and consequently, a high-level explanation of what does the main idea behind task decomposition consist of in each planner.

NOAH considers a network of partially ordered nodes, where each node represents one partially specified task. The planner does not have a clear distinction between primitive and compound tasks, but instead, each task contains some procedural code, declarative code, and links to other tasks. The procedural code denotes the body and the declarative code denotes the effects of a task. A task decomposition consists of evaluating a task body in terms of inserting new tasks in the current task network and applying the task effects. The newly created task network is then checked for potential interactions. In Nonlin, a task network is a collection of nodes, where a node can be considered as a task represented completely in declarative form. A task is represented by a single structure (or schema). This schema describes how a particular task can be accomplished, that is, it defines a special construct (the ‘expansion’ tag) that can contain either a set of tasks, each of which can be further decomposed, or a so-called ‘null expansion’ when the schema represents a primitive task. Given a task network, Nonlin performs a task decomposition such that if a task schema introduces a ‘null expansion’, the task is applied and the process continues with the next task of the task

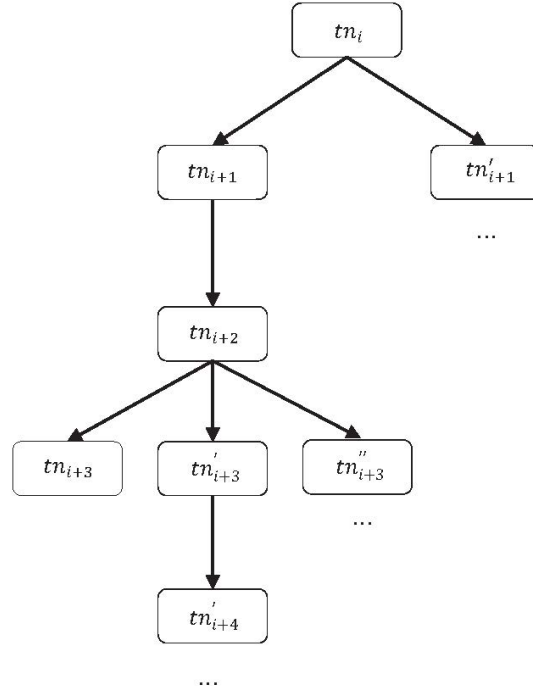


Figure 6: Plan Space

network, and if the schema introduces a list of tasks, the planner expands the task network with the list of tasks and checks for interaction corrections.

SIPE-2 also uses only one structure (called ‘Operator’) to refer to its primitive and compound tasks. Inside this structure, a specific construct (the ‘Plot’ tag) is used to specify, both, an action to be applied or a network of tasks to be further decomposed. If the structure represents a primitive task, the construct contains one task (called ‘PROCESS’ node). If the structure represents a compound task, the construct contains a set of tasks, each of which can require a particular predicate to be achieved (a ‘GOAL’ node), or a task to be performed (a ‘PROCESS’ or a ‘CHOICEPROCESS’ node). A task decomposition consists of the following. Each compound task is decomposed by evaluating a given structure and replacing the task with the set of tasks. The task network is controlled against any newly introduced interactions and corrected accordingly. Each primitive task is copied to the current task network, and task’s (deductive) effects are recalculated based on the context introduced with the modified task network.

O-Plan2 represents and handles tasks similarly as in Nonlin, that is, the planner uses a schema to describe primitive and compound tasks, and given a task network, the planner examines and determines whether there is a schema to be decomposed or a schema to be applied in order to bring the task network into a more detailed level. Finally, UMCP uses explicit notations about primitive (an ‘operator’ tag) and compound (a ‘declare-method’ tag) tasks. The description for the latter type of task includes a list of tasks (the ‘expansion’ tag) and a set of variable binding and ordering constraints associated with the list of tasks (the ‘formula’ tag). Given a network of compound tasks, a task decomposition retrieves all methods associated with a chosen compound task, expands the task network by applying each method to the chosen task, and returns the resulting set of task networks. Each task network is examined for potential conflicts and appropriate resolution steps are initiated.

Constraints

We analyse this concept with respect to other constraint-related concepts, such as the commitment strategy and constraint management.

Commitment Strategy. Plan-based HTN planners take advantage of the least-commitment strategy. However, two deficiencies can be observed. First, except for UMCP, the rest of the planners take a rigid approach of incorporating the commitment strategy into the search mechanism, resulting in tightly coupled and inflexible planning systems. Second, only few planners backtrack on poor decisions, thus not implementing completely the concept of the least-commitment strategy. We confirm the latter observation by examining at which point during the search these planners make decisions and whether the planners backtrack on different choices with respect to variable bindings and task

orderings.

NOAH takes a simple but limited approach. It uses the commitment strategy in such a way that when a task is decomposed, task ordering and variable binding decisions are delayed. NOAH does not backtrack on its choices. The commitment strategy in Nonlin is applied when the planner decomposes tasks in a breath-first manner in the way that variables are bound immediately after they are inserted into the task network, and all constraints are applied before the next decomposition. The planner keeps choices for alternative orderings among tasks and alternative variable bindings. SIPE-2 follows the least-commitment strategy in a depth-first manner by firstly phantomising a goal, which may instantiate variables, then decomposing a task, delaying task ordering decisions and employing a more ‘intelligent’ (with respect to its predecessors and according to the author in [26]) choice of variable bindings. In many cases, the planner avoids backtracking for alternative task orderings and alternative variable bindings. O-Plan2 uses the least-commitment strategy similarly as described in MOLGEN [21]. In particular, the planner deals with a list of entries (called ‘agenda’) that needs to be work out in order the task network to be considered as accomplished. An entry could be decomposing a task, adding ordering or variable binding constraints, etc. In other words, these are pending decisions. For each entry, a priority is calculated and attached to it. Generally, the priority could be expressed as a heuristic value, but in the case of O-Plan2 (as described in [11]) the priority values are fixed by the designer. With this priority, a candidate from the list is chosen and an appropriate modification to the task network is applied. While doing this, the planner takes into consideration alternative variable bindings. UMCP uses the least-commitment strategy to defer task ordering and variable bindings. From the variable bindings perspective, refinement of variable or state constraints either prunes possible values or records variable distinctions. The planner is loosely coupled with the commitment strategy, which enables other commitment approaches to be plugged in. Thus, UMCP is tested with two more commitment strategies, namely an eager-commitment strategy and a dynamic-commitment strategy for which detailed discussions can be found in [23].

The least-commitment strategy is not the only approach that plan-based HTN planners take to manipulate commitments. SIPE-2, O-Plan2 and UMCP provide users with the ability to control commitments interactively. In UMCP, for example, at each decision point, the user is provided a process step to be applied to a task network. The user may accept the suggested step, or can select another step from a list of steps allowed to be applied to the current task network. Another example of interactive commitments is provided below in the description of the phantomisation process in SIPE-2.

Constraint Management: Harmful Interactions. NOAH uses the ‘Resolve Conflicts’ method to identify and resolve deleted-condition interactions. In order to identify these interactions, the planner uses a special table (called ‘Table Of Multiple Effects’) that stores an entry for each expression and its value that is asserted or threatened by more than one task in a task network. For example, the statement (`truck-at t1 l2`)=`true` is a valid entry in the table. By having this information, the planner can recognise a conflict whenever an expression, which is asserted by some task, is threatened by a task that is not the asserting one. As described in Section 2.2, the resolution consists in propagating a constraint stating that the threatened task must be performed first. A double-cross interaction is recognised and resolved by the ‘Resolve Double Cross’ method. NOAH resolves this conflict by identifying which variable is related to the assertion or denial of expressions that led to the conflict. Consequently, the planner finds and inserts other appropriate expressions and/or tasks into the task network ensuring that variables are bound differently at the time of the assertion or denial. The task network with the new expressions and/or tasks must be decomposed again, and in the case of a newly appeared conflict, the planner fails to find a solution.

In contrast to NOAH, Nonlin introduced a rather simple mechanism to deal with interactions. The planner does not treat interactions individually, but provides a unified solution. A causal representation of a task network (called ‘Goal structure’) is used to store information about the task network that would be difficult to extract from the task network itself. With this explicit representation, which equals to the causal link described in Section 2.2, track of task conditions with the points in the network where these conditions are or could be achieved is kept. In the case of interactions, the planner uses its linking process and a table to suggests corrections (similarly to NOAH). These corrections are in the form of linearisations of the network. Linearisations make a particular expression to have correct value for any point in the network to have tasks performed.

In SIPE-2, harmful interactions are detected and resolved by the ‘Solving Harmful Interactions’ method. In fact, the method recognises three cases of harmful interactions. The first case relates to the deleted-condition interaction. As usual, the solution is to order the task network to first accomplish the segment with the precondition and then the corresponding goal. The second case is when a side effect conflicts with a goal (in SIPE-2, a task can have expected and side effects; an expected effect is a fact the planner expects to be true within a task network; a side effect happens otherwise). The solution is to order the task network to first apply the side effect and then the goal. The last case relates to a double-cross interaction. Since there is no ordering that will achieve both goals, SIPE-2 tries to re-achieve one of the goals later, and if it fails, it backtracks and uses another task at a higher level. Resource interactions are

solved by the ‘Resource Conflicts’ method. If a resource-resource interaction is detected, the planner adds ordering constraints as a resolution step. If a resource-argument interaction is identified, a heuristic is used to first order the task (or a part of the task network) using the object as a resource and then the task (or a part of the task network) using the same object as an argument. SIPE-2 uses an additional method to check interactions between planning variables. The ‘Solving a Constraint Network’ method is called at each planning level. In particular, a depth-first search is used to find an appropriate binding for each variable such that no constraints are violated. If no solution is found, the planner prunes the current task network from the search. The planner saves the last found solution for the global constraint network and at each point tries the choice that worked in the previous solution.

In O-Plan2, interactions are handled with the help of constraint managers. A constraint manager is a component that provides efficient support to a higher-level component and does not make any decisions by itself. As such, the constraint manager must maintain information which can be used to prune the search space (if a task network is found to be invalid) or to order search alternatives according to some heuristics [11]. Three constraint managers are used to resolve interactions. The first manager (called ‘TOME/GOST Manager’) enables insertion of conflict-free effects and conditions into a task network. The potential conflicts are resolved with support of the Question-Answering (QA) mechanism. The QA provides answers about the value of some proposition at a particular point in a task network. The second manager (called ‘Plan State Variable Manager’) is used to handle interactions and dependencies among values and variables. This manager maintains the consistency of constraints over these values and variables. Finally, the last manager (called ‘Resource Utilisation Manager’) is used to monitor and provide information about resource levels and resource utilisation.

UMCP uses one resolution method to deal with interactions in a task network. Given a task network to be corrected, the method outputs a set of task networks each of which potentially resolves some of the conflicts in the given task network. UMCP is the only plan-based HTN planner that clearly defines how resolution is accomplished. Since this planner is closest to our definition of plan-based HTN planning, we describe in more details the inclusion of constraint operations in this method. The method incorporates four operations. The first operation (called ‘constraint selection’) determines the constraints to deal with. Given a task network tn , the operation returns k constraint lists that are used to transform the task network into k branches. In a branch i , the i -th constraint list of the form $(c_{i1}c_{i2}\dots)$ is enforced. For each constraint type c_{i1} in the i -th constraint list, the second operation (called ‘constraint enforcement’) produces a set of task networks that make c_{i1} true. Each of the produced task network is further refined by calling this operation associated with c_{i2} , etc., until a set of task networks is produced that make $(c_{i1}c_{i2}\dots)$ true. The next operation is constraint propagation and is adapted as follows. The input to it is a set of task networks from the previous operation. For each of these task networks, the list of constraints the planner has committed to make true (but has not done yet) is examined. It is checked whether some constraints can be enforced at the current level of details in the task network. If such a constraint is found, the previous operation (constraint enforcement) associated with that constraint is invoked. If no constraint is found, the task network is passed to the last operation. Finally, the last operation (called ‘constraint simplification’) consists of evaluation and simplification steps for every type of constraints. If the current level of details in the task network is sufficient to evaluate a constraint, then the task network whose constraints evaluate to false is pruned. Remaining task networks form the output of the resolution method. If the current level of details is not sufficient, then the evaluation step returns either the constraint itself or a simplified version of it [12].

Constraint Management: Helpful Interactions. NOAH employs the ‘Use Existing Objects’ method to identify placeholder-replacement interaction, and thus to replace a formal object with an actual value. In NOAH, the use of this method may imply merging some tasks in a task network in terms of task reordering and partial linearisation of the task network. The planner uses the ‘Optimise Disjuncts’ method to handle situations when disjunct optimisation is possible. The method builds planner’s special tables (a table refers to ‘Table of Multiple Effects’) for each choice among disjuncts, examines these tables and looks for a choice that will substantially minimise the number of tasks in the overall task network. If such disjunct exists, it is selected and the rest of the disjunction is ignored. Furthermore, NOAH identifies phantomisation by using the ‘Eliminate Redundant Preconditions’ method. When redundant goals are found, the method simply eliminates them. The rationale behind this action is to avoid redundant processing in further planning levels, and to save memory space. In Nonlin, the phantomisation is handled by the linking process exactly as described in Section 2.2.2. Furthermore, SIPE-2 recognises phantomisation by using the ‘Goal Phantomisation’ method. Since the planner is not able to solve this situation correctly by itself, it provides the user with two options to phantomise goals, particularly by using variable bindings or linearisations [9]. In the case of variable bindings, the user has three choices, namely: bind a variable whenever possible, never post constraints, or bind a variable only when there is one possible binding to accomplish the goal. In the case of linearisations, the user has also three choices: never add ordering constraints for the purpose of phantomisation, add ordering constraints when no other constraints are required to accomplish the phantomisation, or add ordering constraints when there is a parallel branch with an

Table 1: Resolution Methods for Task Interactions in Plan-Based HTN Planners

Interaction	NOAH	Nonlin	SIPE-2	O-Plan2	UMCP
Deleted-condition	Resolve Conflicts	Linking Process	Solving Harmful Int.	TOME/GOST Man.	Resolution Method
Double-cross	Resolve Double Cross	×	Solving Harmful Int.	×	×
Resource	×	×	Resource Conflicts	Resource Util. Man.	×
Placeholder replacement	Use Existing Objects				
Phantomisation	Eliminate Redun. Prec.	Linking Process	Goal Phantomisation	Question Ans.	Domain Method
Disjunct optimisation	Optimise Disjuncts				×

effect that has only one possible instantiation for accomplishing the phantomisation. O-Plan2 uses existing tasks in the network to satisfy some state constraints at any point the network. O-Plan2 is supposed to handle all situations that need to be phantomised in Nonlin.

In UMCP, the phantomisation is performed in a different way. If some predicate q of a (goal) task t is already true, an empty task network represents its accomplishment. For that purpose, for each such task, there is an explicit method that includes a dummy primitive task t_d with no effects, and the condition that the predicate q should be true immediately before the t_d .

Summary Table 1 summarises and classifies resolution methods with respect to the task interaction they solve. If a cell contains ‘X’, it means that the planner does not need to and does not handle the respective interaction. If a cell is empty, then it means that the information was not available from the literature.

Constraint management involves heavy computation and makes plan-based HTN planners face computational difficulties. It is hard to determine whether a given predicate is true or not at a given point in a task network. Nonlin, for instance, uses its QA mechanism to determine the value of such a predicate. However, the answer might not be deterministic due to the partial order of the tasks, that is, it may answer with ‘yes’, when the predicate can have a particular value, ‘no’, when the predicate can not have the particular value, or ‘maybe’, when it may have the particular value, some other value or undefined. SIPE-2 employs a truth criterion to reason over the truthfulness of a formula. As the formula is formed by a conjunction of predicates, the formula truth criterion is further reduced to a rather complex predicate truth criterion. Moreover, SIPE-2 restricts the partial orderings (nonlinearity in SIPE-2’s term) for the purpose of circumventing the NP-completeness of the truth criterion [9]. O-Plan2’s truth criterion is an extended version of Nonlin’s QA. For example, if the answer is ‘maybe’, QA provides an alternative set of strategies to ensure the truth of a predicate. UMCP extends Chapman’s Modal Truth Criterion (MTC) [34], which is similar to QA, to also consider compound tasks. UMCP uses the extended MTC to evaluate the state constraints and it runs in quadratic time [12].

Explicit Conditions

We summarise and classify explicit conditions that plan-based HTN planners employ in Table 2. Similarly to Table 1, a cell containing ‘X’ denotes that a planner does not support the respective condition. We observe that the idea of explicit conditions is initiated by Nonlin. The planner supports four types of conditions. A supervised condition is always inserted explicitly in a network (through a ‘GOAL’ node). External conditions are considered by the planner only when all “compound” tasks are decomposed into “primitive” ones. In SIPE-2, a supervised condition ensures that the main effect of a task remains true until the condition holds. An external condition defines that the goal will be achieved by a possibly parallel action and phantomised by an ordering link [35]. Conditions play a special role in O-Plan2, since the planner does not consider any notion of a goal (a ‘GOAL’ node) [11]. Instead, O-Plan2 takes advantage of the achieve condition as the only one in which insertion of new tasks is allowed. In fact, this condition can be expressed in two ways. Firstly, the ‘achieve at N’ condition supports insertion of tasks in a task network without temporal restrictions. Secondly, the ‘achieve at N after <time point>’ poses temporal restrictions on the parts of the task network that could satisfy this condition. The latter condition is a generalisation of the achieve condition in NOAH, Nonlin and SIPE-2 [36]. In contrast to Nonlin, UMCP handles external conditions at higher planning levels to prune the search space. External conditions are represented as state constraints which means that the planner will look for variable bindings or task ordering that make the conditions true, but the planner does not allow an establishment of the conditions by insertion of new tasks into a task network, or by task decomposition. Filter conditions are represented as state constraints as well, and used by the resolution method to prune inconsistent

Table 2: Condition Types in Plan-Based HTN Planners

Condition	NOAH	Nonlin	SIPE-2	O-Plan2	UMCP
Supervised	precondition	supervised	protect-until	supervised	goal task
External	×	unsupervised	external-condition	unsupervised	state constraints/external
Filter	×	use-when	precondition	only_use_if	filter
Query	×	×	×	only_use_for_query	×
Compute	×	×	×	compute	×
Achieve	×	×	×	achieve at N	×
	goal	goal	goal	achieve at N after <time point>	goal task

task networks. If filter conditions are not affected by any task, then it suffices to check the initial state to evaluate them. The flexibility of UMCP has been proven again by extending the planner to reason about implicit external conditions [37]. Thus, instead of being specified explicitly, external conditions occur as a result of the structure of the domain knowledge, and are detected by examining the domain knowledge.

2.4 State-Based HTN Planning

We complement Definition 2 and Definition 3 of the theoretical framework as follows.

Definition 10 (Task network). A *task network* tn is a pair $(T, \prec)$, where

- T is a finite set of tasks
- $\prec$ is a strict partial order on T (irreflexive, transitive, and asymmetric)

Definition 11 (Method). A *method* m is a triple $(c(m), pre(m), tn(m))$, where $c(m)$ is a compound task, $pre(m) \in 2^Q$ is a precondition, and $tn(m)$ is a task network. The subsets $pre^+(m)$ and $pre^-(m)$ denote positive and negative precondition of m , respectively.

A method m is applicable in state s , if and only if $pre^+(m) \subseteq s$ and $pre^-(m) \cap s = \emptyset$. Applying m to s results in a new task network.

Definition 12 (Decomposition). Let m be an applicable method in s and $tn_c = (T_c, \prec_c)$ be a task network. Method m decomposes tn_c into a new task network tn_n by replacing task t , written $tn_c \xrightarrow[s, t, m]{D} tn_n$, if and only if $t \in T_c$, $t = c(m)$ and

$$tn_n := ((T_c \setminus \{t\}) \cup T_m, \prec_c \cup \prec_m \cup \prec_D) \text{ where} \\ \prec_D := \{(t_1, t_2) \in T_c \times T_m \mid (t_1, t) \in \prec_c\} \cup \{(t_1, t_2) \in T_m \times T_c \mid (t, t_2) \in \prec_c\}$$

Definition 13 (Decomposition). Let m be an applicable method in s and $tn_c = (T_c, \prec_c)$ be a task network. Method m decomposes tn_c into a new task network tn_n by replacing task t , written $tn_c \xrightarrow[s, t, m]{D} tn_n$, if and only if $t \in T_c$, $t = c(m)$ and

$$tn_n := ((T_c \setminus \{t\}) \cup T_m, \prec_c \cup \prec_m \cup \prec_D) \text{ where} \\ \prec_D := \{(t_1, t_2) \in T_c \times T_m \mid (t_1, t) \in \prec_c\} \cup \{(t_1, t_2) \in T_m \times T_c \mid (t, t_2) \in \prec_c\}$$

A similar model of HTN planning as this one is presented in [33] where the authors define a progression problem space. The space is a directed graph in which pairs of state and task network are vertices, and a progression from one pair to another is an outgoing edge. We take a slightly different approach in which a state is a vertex, and a task decomposition maps to the same state where the corresponding method is applicable, and operator application leads to a successor state.

Definition 14 (State space). Given a state-based HTN planning problem $\mathcal{P}$, a *state space* SG is a directed graph (V, E) if and only if $s_0 \in V$, and there is a state s_i and $t_k \in tn$ such that

- if t_k is primitive, then $s_i \xrightarrow[t_k]{t_k} s_{i+1}$ such that $k = i + 1$, $s_i, s_{i+1} \in V$ and $(s_i, s_{i+1}) \in E$; or
- if t_k is compound, then $tn \rightarrow_D tn'$ is a self-transition such that $s_i \in V$ and $(s_i, s_i) \in E$.

The HTN state space is illustrated in Figure 7.

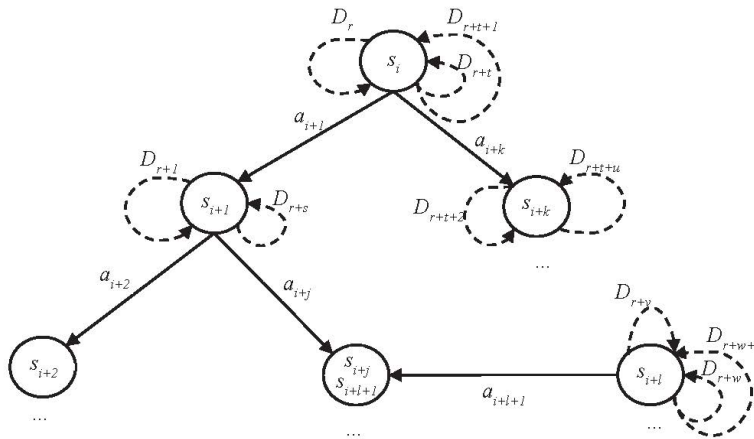


Figure 7: State Space

2.4.1 Review of State-Based HTN Planners

We explore chronologically state-based HTN planners in terms of the concepts described in Section 2.2. We show that these planners employ a simpler approach to planning compared to plan-based HTN planners. The concept choices are easy to understand and indeed influence the structure of the space.

Task Decomposition

SHOP2 and SIADEx use a specific construct (‘method’ and ‘task’, respectively) to represent a compound task amenable to several *ways* of accomplishment. Each way (‘method’ in SIADEx) contains applicability conditions and a task network. In both planners, the set of ways can be seen as an if-then-else representation, that is, the planners select the first way whose if-statement (preconditions) holds in the current state. Thus, given a compound task, a task decomposition evaluates the preconditions of task’s associated ways, and chooses the first way applicable in the current state to expand the existing task network. Two observations are in order. First, recall that in Section 2.2.1 we stated that the task decomposition makes a non-deterministic choice of which method to use for the decomposition. However, in the case of both planners, the choice is controlled, that is, the first method from the if-then-else representation that is applicable in the state is chosen. Second, the newly composed network does not need to be checked for corrections.

Among state-based HTN planners, SHOP uses the ordered task decomposition, while SHOP2 the unordered task decomposition (by using ‘ordered’ and ‘unordered’ tags to denote totally and unordered task networks, respectively). In SHOP2, effort is made to bring the planner closer to the partially ordered task decomposition by allowing partial restrictions on the order of unordered task networks. In particular, if some method with a task network tn has a task t_i that begins with the ‘immediate’ tag, then the planner knows that it must do t_i immediately after t_{i-1} finishes, without trying to interleave other tasks between t_{i-1} and t_i . SIADEx, on the other hand, follows the partially ordered task decomposition. Tasks in a network can be ordered, unordered and in parallel.

Constraints

Similarly to the previous section, we analyse this concept with respect to the commitment strategy and constraint management of state-based HTN planners.

Commitment Strategy. State-based HTN planners employ the early-commitment strategy. Thus, they know the current state at each step in the planning process. In fact, the state is transformed only by applying an action to it. With respect to task ordering, SHOP2 and SIADEX greatly benefit from adopting the forward chaining approach in which chaining of actions is achieved by imposing total order on (some) plan actions. The total ordering ensures that neither the current action to be added to the plan can interfere with some earlier action’s preconditions or effects, nor a later action can interfere with current action’s preconditions or effects. With respect to variable bindings, all variables are instantiated when task preconditions are evaluated and an action is applied. Thus, all actions in a plan are fully specified. The evaluation of preconditions implies a list of all possible sets of variable bindings that satisfy these preconditions in the current state. If some task later fails, both planners backtrack on other alternatives according to the variable bindings left in the list, or maybe to some criterion specified in the definition of the task. Considering the truth value of the preconditions, SHOP2 and SIADEX prune inappropriate tasks from the search space.

Constraint Management. By taking the early-commitment strategy into consideration, we could conclude that state-based HTN planners avoid task interactions altogether. However, this statement is not entirely correct. Table 3 suggests that state-based HTN planner avoid most interactions. In SHOP2, for example, a deleted-condition interaction may arise due to the process of interleaving tasks. The planner is able to solve this situation under a rather restricting assumption, that is, it requires a specification of ‘protection’ conditions in the effects of operators. A protection request enforces the planner from deleting conditions, and a protection cancellation allows the planner to delete these conditions. For example, consider our running-example domain. To give knowledge to SHOP2 that after driving the truck *t* from location *l_f* to location *l_t*, the truck should stay at *l_t* until a box is loaded into *t*, we need to modify the operator *drive* by adding a protection request, denoted by *:protection*, for *truck-at(t,l_t)*, i.e., (*:protection truck-at(t,l_t)*). A protection cancellation can be added into the negative effects of the *load-truck(t,b,l)* operator in a similar manner. SIADEx needs a more powerful mechanism to accomplish planning and handle interactions that may arise in partially ordered task networks. The planner uses a causal structure of tasks and task networks. Constraint satisfaction checks the consistency of task networks (and the solution) based on the causal structure, and constraint propagation is used to post constraints, if necessary.

State-based HTN planners include a certain form of phantomisation. In SHOP2 and SIADEx, similarly as in UMCP, the phantomisation of a task is explicitly encoded in the domain knowledge. The planners handle the phantomisation of a rather recursive task by taking into account an alternative decomposition which explicitly encodes a ‘do-nothing’ operation. Therefore, one could say that the planners lack the ability to infer such situations by themselves. This issue is addressed in [29], where authors extend the JSHOP2 planner [38], that is, a Java implementation of SHOP2, to reason about the conditions that are already achieved without the need of explicit specification into the domain knowledge at the expense of spending negligible additional planning time.

For example, consider our running-example domain. Let say that the truck *t1* should take the box *b1* from a location *l3* to a location *l1*, and from there to a location *l2*, and from *l4* the box should be transported by a plane *p* to the final location *l4*, under the assumption that the locations are adjacent as described, and that *l1*, *l2* and *l3* are in a same city, while *l4* is in another city. For that purpose, in the scope of the *deliver* task, we can write a method with several branches. One decomposition could describe how to transport a box by a plane, another decomposition could describe how to transport a box with a truck, and, finally, the last decomposition describes explicitly that the transportation of the box is finished since the box is already at the desired location.

```
task:      deliver(b,lf,lt)
way 1:     transport a box with a plane
precondition: plane-at(p,lf),adjacent(lf,ln),box-at(b,lf),
              in-city(lf,cf),in-city(ln,cn),different-city(cf,ct)
task network: <load-plane(b,p,lf),fly(p,lf,ln),
              unload-plane(b,p,ln),deliver(b,ln,lt)>

way 2:     transport a box with a truck
precondition: truck-at(t,lf),adjacent(lf,ln),box-at(b,lf),
              in-city(lf,cf),in-city(ln,cn),same-city(cf,cn)
task network: <load-truck(b,t,lf),drive(t,lf,ln),
              unload-truck(b,t,ln),deliver(b,ln,lt)>

way 3:     no transport needed
precondition: box-at(t,lt)
task network: <>
```

Explicit Conditions

State-based HTN planners do not share the strong need for explicit conditions with plan-based HTN planners, as shown in Table 4. The whole reasoning power of SHOP2 and SIADEx is encapsulated in the preconditions of both primitive and compound tasks, thus they do not require other explicit domain knowledge. In the scope of preconditions, however, SHOP2 enables several types of computations, such as invocations of external knowledge resources by using the ‘Call’ condition. SIADEx also supports complex computations by incorporating complete (Python-based [39]) procedures in the domain. External conditions are modelled in a similar fashion.

Table 3: Resolution Methods for Task Interactions in State-based HTN planners

Interaction	SHOP2	SIADEx
Deleted-condition	Protection condition	Constraint propagation
Double-cross	×	×
Resource	×	×
Placeholder replacement	×	×
Phantomisation	Domain method	Domain method
Disjunct optimisation	×	×

Table 4: Condition Types in State-based HTN Planners

Condition	SHOP2	SIADEx
Supervised	×	×
External	Call	Arbitrary code
Filter	precondition	precondition
Query	×	×
Compute	Call	function
Achieve	×	×

3 Analysis of HTN Planning and HTN Planners

So far we categorised HTN planners based on the space they search in. This categorisation highlights the common search-related features among planners, though we have two more reasons for analysing the planners and HTN planning. First, this categorisation does not cover the capabilities of HTN planners with respect to the level of knowledge these planners require, which expressiveness constructs the planners support, and what is the performance of these planners. The second reason lies in some of the implicit assumptions made about HTN planners, that is, claims and beliefs accepted for granted and without evidence. These include the “sophistication” of domain knowledge provided to HTN planners, the expressive power of HTN planning in theory and practice, HTN planners being fast and scalable, and being HTN planning very suitable for and most applied to real-world problems.

To this end, we provide the third piece of our framework pie, that is, we develop an analytical framework to collect and organise studies on HTN planning and planners. Then, we apply exploratory research to examine diversity and similarity of HTN planning within their category and between categories, and comparative research to make sense of a range of cases. In this way, we believe that statements about domain knowledge, practical expressiveness, performance, and applicability can be made in a neutral and evidence-oriented way.

3.1 Analytical Framework

We develop an analytical framework to collect and organise data about HTN planners. The framework directs us on where to look and what kind of properties to look for but without making specific hypotheses about relationships among properties. The framework consists of five main elements, namely, domain authoring, expressiveness, competence, performance, and applicability. Each element, the motivation for its inclusion in the framework, and its corresponding properties are defined in the following subsections.

3.1.1 Domain Authoring

An interesting perspective on HTN planners says:

“[Compared with classical planners,] the primary advantage of HTN planners is their sophisticated knowledge representation [and reasoning capabilities] [4].”

Two remarks are in order. First, there is uncertainty in the meaning of “sophisticated”. Does it refer to the complexity, richness or some other attribute of the representation? For now, let us assume that it refers to the so-called “knowledge-rich” representation [40]. The second remark is on HTN planners taking advantage of the use of knowledge-rich encoding. On the one hand, this could be correct, if we consider that these planners improve their performance (over classical planners) thanks to their domain knowledge [41]. On the other hand, why are HTN planners in advantage if we do not know at what expense, in terms of encoding effort, we obtain that improvement?

We define *domain authoring* the process of domain knowledge performed by a domain author. What we are interested in, in this process, is the relative effort needed to formulate such domain knowledge for an HTN planner. However, the community has not yet found a way or measures to provide an objective answer to this type of questions, thus the answer remains rather anecdotal. It is ambiguous and difficult to define an answer especially because it directly depends on the capabilities and experience of the domain author with respect to the understanding of the underlying planning system and the expertise for the respective domain.

With the observation that the knowledge-rich representation is a requirement for HTN planners, we give a flavour of HTN planners’ requirements by taking a model of the well-known and overused domain of block world as described for each planner, and inspecting each model from two aspects. First, we take the same task of each domain model and analyse closely what needs to be encoded. Second, we give a broader view of each domain model by quantifying its content with respect to knowledge symbols, keyword symbols, and domain elements.

3.1.2 Expressiveness

We tackle expressiveness from two perspectives. The first one is the formal expressiveness of HTN planning language, and requires formal semantics of the language that completely determines what the language can express. This issue has been a subject of discussion for some time, resulting in a number of studies on expressiveness of HTN planning, such as [42, 31, 43, 40, 44, 12]. The most comprehensive analysis is provided in [45], where the expressiveness of HTN planning language is analysed from a model-theoretic, operational and computational aspect. In each aspect, the expressiveness of HTN planning is compared to the one of STRIPS-like planning. Thus, we gain a perspective in theoretical expressiveness by summarising the findings in [45]³.

The second perspective is more practical, thus suggesting a relaxed definition of the expressive power. In this case, *expressiveness* of a language is determined by the breadth of what the language can represent and communicate. The breadth includes assessment of the description languages of HTN planners with respect to their formal system, preference support, etc. Unfortunately, there is no common planning language for HTN planners. The idea of standardising a planning language is introduced with the Planning Domain Definition Language (PDDL) [46] in 1998 for the purpose of the International Planning Competition (IPC), rather late with respect to the history of, above all, plan-based HTN planners. Although in the first version of PDDL there was an attempt to formalise a common syntax compatible to HTN planners, the idea was discarded with version 2.1 of PDDL [47] due to the immense differences between planners.

Since we are still interested in what HTN planners can express in practice, we explore the language expressiveness of each HTN planner from three aspects. First is the use of first-order logic, where we want to determine the support for different logical operators. The second aspect is on the use of quality constraints, where we want to determine the support for sort hierarchy, extended goals, and preferences. We define each as follows. A *sort hierarchy* expresses characteristics of an object in a type hierarchy (similar to typing in PDDL). The basic idea of *extended goals* is to express a planning objective in a way that its satisfaction could be on a part or on the whole trajectory of the plan, and not just in the final state. A *preference* is a condition on the plan trajectory that some user would prefer satisfied rather than not satisfied, but would accept if the condition might not be satisfied [48].

The third aspect is on the use of scheduling, where we want to determine the support for resource and time management. By *resource* we mean an object of limited availability within a planning problem. We define and consider when analysing the planners a resource hierarchy, as shown in Figure 8. Each element of the hierarchy is defined as follows. A reusable resource is a resource that can be used again after its initial use. A shared reusable resource can be shared among several tasks at the same time, while an exclusive reusable resource cannot be used by two tasks in parallel. A consumable resource is a resource that is usable only a limited number of times. A consumable resource can be replenished or not. If the resource cannot be restored after the use of the set amount, it is called disposable consumable resource. Otherwise, if the resource amount can be topped up, it is called renewable consumable resource. Time is considered as usual, that is, a consumable resource that cannot be reproduced. We are interested in how HTN planners acquire and handle temporal information. In the simplest case, we are curious about three explicit temporal forms of a task, that is, the start of a task, the end of a task, and the duration of a task.

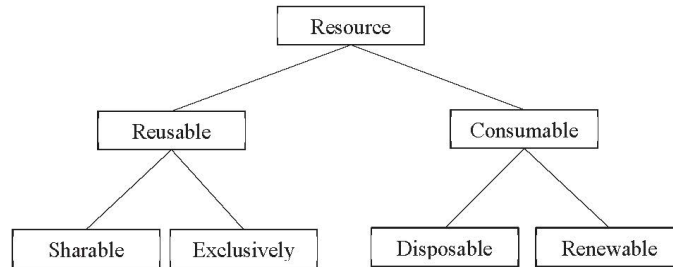


Figure 8: Resource Hierarchy

3.1.3 Competence

We define *competence* as the ability of an HTN planner to accomplish or address a specific feature, such as domain dependence, fault tolerance, completeness, and soundness:

- *Domain dependence* defines the class of a planner based on whether (and in what way) the planner can be configured to work in different domains [4]. The following classes exist: 1) domain-specific planners are composed

³We assume that the reader is familiar with model-theoretic, operational and computational-based expressiveness.

for particular domain and probably unsuitable for any other domain; 2) domain-independent planners have a general planning engine that works in any domain, while the input is a planning problem to be solved; 3) domain-configurable planners are provided with a domain-independent planning engine, but the input is a domain-specific knowledge.

- *Solution flexibility* defines the ordering of actions in the solution plan. We say that a solution is flexible if it is partially ordered.
- *Complexity of the search mechanism* defines the parts that form the mechanism, and the easiness to adapt or modify the mechanism. This includes an algorithm with or without backtracking points (e.g., a choice among few tasks, a choice on variable bindings and task ordering), use of heuristics to help the search, search control interactively performed by a user, evaluation of preconditions, and flexibility to include other automatic algorithms.
- *Fault tolerance* defines the property of a planner to recognise, react appropriately, and continue operating in the event of a failure. We are interested in monitoring plan executions in order to identify problems, and replanning by modifying the existing or generating a new plan in order to solve such problems.
- Given an HTN planning problem, an HTN planner is *complete* if it always finds a solution to the planning problem when such a solution exists.
- Given an HTN planning problem, an HTN planner is *sound* if every answer it gives is a correct solution to the planning problem.

3.1.4 Performance

Similarly to expressiveness, we are interested in performance from two perspectives. The first perspective refers to the theoretical boundaries of HTN planning. This aspect is already addressed in [45], where time and space complexity of HTN planning are analysed. The computational problem, called plan existence, poses the following question: *given an HTN planning problem, is there a plan that solves the problem?* In order to properly understand the results of analysis, the following possible settings should be taken into account.

- The sets of operators and methods can be provided in two ways. The sets can be a part of the *input*, or they can be *fixed* in advance, that is, the tasks are allowed to contain decompositions corresponding only to predicates in the initial state.
- A compound task can be defined in several ways. A compound task is without any restriction (*yes*); a *regular* task in task networks, that is, at most one compound task followed by all primitive tasks; an *acyclic* task, that is, a task can be decomposed to only a finite depth; and, finally, compound tasks are not allowed at all (*no*).
- A task network containing primitive and compound tasks as defined in the previous point can be *totally ordered* or *partially ordered*, whenever there is a total or partial order among those tasks, respectively.
- Variables can be allowed or not in the planning problem.

The second perspective gives insights into the practice of testing of HTN planners. We are interested in the runtime and scalability results of each HTN planner. We say that a planning system is *scalable* if it is capable to cope and suitably perform under a varying size of planning problems. Anything but easy is to define dimensions that could measure the size of a problem, nevertheless, scalability is highly desirable in practical settings with an increasing and large number of facts about the state, a large number of users, or a large number of tasks. We are interested in how good planners scale relative to one another assuming increasingly difficult problems. As for *runtime*, we are interested in pairwise comparisons between HTN planners with respect to the amount of time they spend on the same sets of problems.

3.1.5 Applicability

Applicability concerns the use of planners in real application domains. It appears to be orthogonal to previous categories. We have two reasons for the inclusion of this element in the framework. First, we strongly believe that the ultimate objective of research on automated planning must be application of planners in a variety of real domains: oil spills [49], spacecraft assembly [50], microwave manufacturing [51], smart spaces [52], and Web service composition [53], to name a few prominent examples. Second, HTN planning is promoted as the most applied automated planning technique of real-world problems [54], however, mostly referring to the applications of SHOP2. Thus, we want to see 1) whether HTN planning contributes towards the aforementioned objective, and 2) what is the status of applicability of other HTN planners.

3.2 Outcome

For each element of the analytical framework, we explore the literature and apply descriptive research to understand the respective property. In two cases, we provide theoretical and practical perspectives of a framework element. Where possible, we also show comparison of HTN planners. In some cases, we aggregate the data on planners in tabular form. We use the following common notation for all tables. The ‘✓’ denotes that a planner supports the respective property, the ‘✗’ indicates that a planner does not support a particular property, and an empty cell denotes that it is unknown from the literature whether the planner is able to deal with a given element. There are rated properties, where the rate ranges from ‘★’, denoting limited support for the given property, to ‘★★★’, indicating extended support.

3.2.1 Domain Authoring

Figure 9 shows the description of the ‘put-on’ task provided to each HTN planner except SIADEx for which we were not able to find neither a task nor a domain description. We start by describing NOAH’s task which specifies that three statements should be evaluated in sequence (line 4, 10 and 12). The first statement is handled by evaluating lines 5 and 7, which cause new tasks to be added. If a predicate (e.g., ‘(cleartop \$x)’) is not true, we should expect that a task would be added to achieve the predicate, otherwise some form of a ‘do-nothing’ is assumed. The statement in line 10 is evaluated analogously. For the last statement in line 12, we need to be aware that it deletes a predicate from the (global) state, but also inserts that predicate in the add list of some dummy task at the current level (of planning).

Planner	Description	Planner	Description
NOAH	<pre> 1 (puton 2 (qlambda 3 (on <-X <-Y) 4 (pand 5 (pgoal (clear \$x) (cleartop \$x) 6 apply (clear)) 7 (pgoal (clear \$y) (cleartop \$y) 8 apply (clear)) 9) 10 (pgoal (put \$x on top of \$y) 11 (on \$X \$y) apply nil) 12 (pdeny (cleartop \$y)) 13) 14) </pre>	Nonlin	<pre> 1 actschema puton 2 pattern <<put \$*x on top of \$*y>> 3 conditions 4 holds <<cleartop \$*x>> at self 5 holds <<cleartop \$*y>> at self 6 holds <<on \$*x \$*z>> at self 7 effects 8 + <<on \$*x \$*y>> 9 - <<cleartop \$*y>> 10 - <<on \$*x \$*z>> 11 + <<cleartop \$*z>> 12 vars x undef y undef z undef; 13 end; </pre>
SIPE-2	<pre> 1 operator: puton 2 arguments: block1, object1 is not block1; 3 purpose: (on block1 object1); 4 plot: 5 parallel 6 branch 1: goals: (clear object1); 7 branch 2: goals: (clear block1); 8 end parallel 9 process 10 action: puton.primitive; 11 arguments: block1, object1; 12 resources: block1; 13 effects: (on block1 object1); 14 end plot end operator </pre>	O-Plan2	<pre> 1 schema puton; 2 vars ?x=undef, ?y=undef, ?z=undef; 3 expands {put ?x on top of ?y}; 4 only_use_for_effects 5 {on ?x ?y}=true, 6 {cleartop ?y}=false, 7 {on ?x ?z}=false, 8 {cleartop ?z}=true; 9 conditions 10 only_use_for_query {on ?x ?y}=true, 11 achievable {cleartop ?y}=true, 12 achievable {cleartop ?x}=true; 13 endschema; </pre>
UMCP	<pre> 1 (operator puton (x y) 2 :pre ((clear x)(on x table)(clear y)) 3 :post ((~on x table)(on x y)(~clear y)) 4) </pre>	SHOP2	<pre> 1 (:operator (!puton ?x ?y) 2 ((clear ?x) (on-table ?x) (clear ?y)) 3 ((clear ?y) (on-table ?x)) 4 ((on ?x ?y)) 5) </pre>
SIADEx	<pre> 1 (:action puton 2 :parameters (?x ?y - block) 3 :precondition (and (grasping ?x)(clear ?y)) 4 :effect (and (not (grasping ?x)) 5 (not (clear ?y))(clear ?x) 6 (on ?x ?y)(handempty)) 7) </pre>		

Figure 9: The ‘put-on’ Block-World Task in HTN Planners

In contrast to NOAH, the encoding of Nonlin is more clear. In Nonlin, the ‘put-on’ task contains three filter conditions (line 3 to 6). The first two filter conditions state that before applying the task two blocks must be clear. The third filter condition is used to bind some variable with what is on top of the variable from line 3. If all of these conditions hold, the effects of the task are applied (line 7 to 11). With SIPE-2 things are getting complicated again. Its task contains the following expressions of interest. Line 3 specifies the goal that this task can achieve. In the element of line 5, a task network of two parallel tasks is contained with the purpose of achieving some predicates. The

element in line 9 specifies the action that should be used in order for the ‘put-on’ task to be accomplished. The task contains no information about when a block is clear or not, or when a block is not on another block. This information would be inferred by planner’s deductive theory. O-Plan2’s task is very similar to the one of Nonlin. Line 3 specifies the task network used to accomplish the task, that is, a single action. The expressions in line 4 to 9 are the actual effects of the action. On the other hand, three conditions must be satisfied in order for these effects to be applied. The conditions in line 9 and 11 might be achieved by other tasks during the planning process. The condition in line 12 is used to bind a variable for the purpose of specifying the effects.

The task descriptions for UMCP, SHOP2 and SIADEx differ only in the notation, but specify the same meaning. All tasks, that is, operators contain simple applicability preconditions (line 2 in SHOP2), and effects (line 4) as a postcondition in UMCP, and as a delete and an add list in SHOP2 (line 3 and line 4, respectively). However, beside the representational simplicity, the power of these operators is much weaker than the tasks of the previously discussed planners. The operators cannot handle situations where some block is above another one or when a block is not clear. The solution to achieve fairly equal functionality is to write methods that describe all possible situations.

Figure 10a gives a quantitative perception of the size of the ‘put-on’ task descriptions from Figure 9 in terms of the number of symbols. At first glance, UMCP and SHOP2 seem to require smaller ‘put-on’ descriptions, but looking at the number of symbols at the domain level, as shown in Figure 10b, SIPE-2 has the largest domain description, however, almost half of it belongs to keyword symbols. On the other hand, SHOP2 has slightly smaller domain description than SIPE-2, but the number of keyword symbols is negligible, which means that the rest of the symbols represent the actual domain knowledge. Figure 10c makes it even more clear. UMCP and SHOP2 need more tasks in total than the rest of HTN planners in order to successfully perform planning in the domain. In fact, SHOP2 needs 13 operators and methods in total, and 6 axioms to do planning, while O-Plan2, for example, needs only three tasks in total.

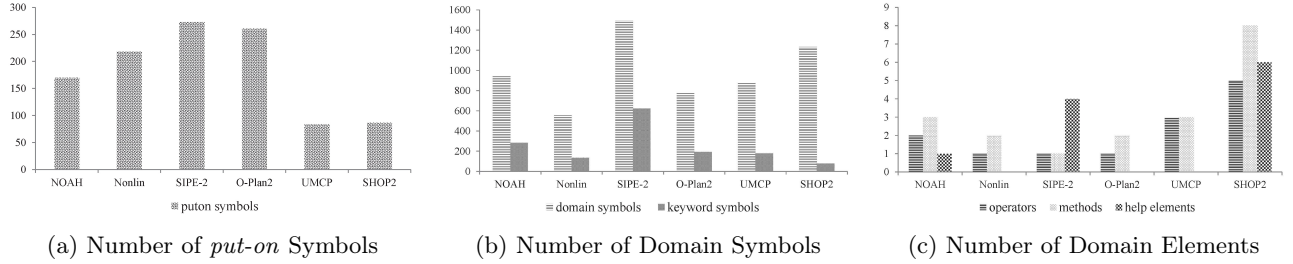


Figure 10: Quantitative Perception of the ‘put-on’ Task Description and Block-World Domain

Interpretation

The analysis of tasks shows, on the one hand, that a prerequisite to author domain knowledge for most plan-based HTN planners is the comprehension of their underlying systems, such as expectations of what the system would do in a particular situation. On the other hand, a domain author does not need to have a priori familiarity with a state-based HTN planner, but the author must provide quite powerful and elaborate domain configuration. This is supported by the analysis of results shown in Figure 10. Additional evidence to the latter observation is the criticism of SHOP2 planner that it is a problem-solving programming language rather than a planner [55]. Finally, Figure 10b indicates the richness of domain descriptions with knowledge.

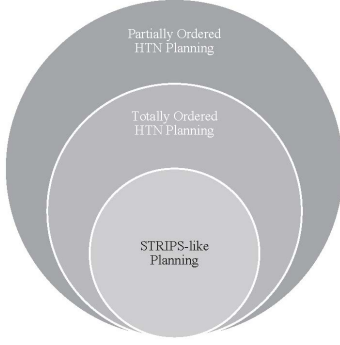
3.2.2 Expressiveness

Expressiveness in Theory

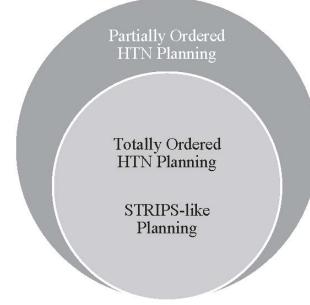
Figure 11a depicts the model-theoretic expressiveness. From this aspect, the HTN language is strictly more expressive than the STRIPS language, but totally ordered HTN planning is less expressive than partially ordered HTN planning, and strictly more expressive than STRIPS-like planning [43]. An HTN planning problem (with totally or partially ordered task networks) can be transformed into a STRIPS-like planning problem, but the converse is not true. On the other hand, a totally ordered HTN planning problem can be transformed into a partially ordered HTN planning problem, but the converse is not true.

Figure 11a shows that the operational aspect has the same expressiveness hierarchy as the model-theoretic aspect. That is, HTN planning is strictly more expressive than STRIPS-style planning, and totally ordered HTN planning is strictly between STRIPS-like planning and partially ordered HTN planning.

Figure 11b depicts the computational-based hierarchy. Once more, HTN planning is strictly more expressive than STRIPS-like planning. In particular, there is a (polynomial) transformation from STRIPS-like planning to



(a) Hierarchy of model-theoretical and operational expressiveness



(b) Hierarchy of computational-based expressiveness

Figure 11: Hierarchies of expressiveness

Table 5: Practical Expressiveness of HTN Planners

Property	NOAH	Nonlin	SIPE-2	O-Plan2	UMCP	SHOP2	SIADEx
Conjunction	✓	✓	✓	✓	✓	✓	✓
Disjunction	preconditions	×	preconditions	constraints	preconditions	preconditions	preconditions
Negation	effects		preconditions effects	effects	constraints effects	preconditions	preconditions
Implication	×	×	×	×	×	preconditions	
Existential Q.			preconditions conditions		constraints		
Universal Q.	×	×	effects	×	×	preconditions effects	preconditions effects
Sort Hierarchy	×	×	★★★	★	×	×	
Extended Goals	×	×	×	×	×	×	★
Preferences	×	★	★★	★	★	★★★	×
Resource	×	×	★★	★★★	×	×	×
Time	×	×	★	★★★	×	★	★★★

HTN planning, but there is no computable transformation from HTN planning to STRIPS-like planning. Intuitively, HTN elements can represent computationally more complex problems than STRIPS-like operators. However, these results are true when partially ordered task networks are allowed. In fact, totally ordered HTN planning is at the same level of expressiveness as STRIPS-like planning, but significantly less expressive than partially ordered HTN planning. This is because totally ordered HTN planning avoids interleaving of tasks from different compound tasks.

Expressiveness in Practice

Table 5 illustrates the expressiveness properties of the planning languages of HTN planners. SIPE-2, UMCP, SHOP2 and SIADEx employ first-order logic in their preconditions with some restrictions, while disjunction is not allowed in the effects of tasks. Thus, task preconditions are more expressive than the task effects. Except Nonlin and SHOP2, which use deletion of a predicate, other HTN planners use negation of a predicate in the effects. SHOP2's language supports about the level 2 of the PDDL version 2.1, and allows declarations of implications in the task preconditions and universally quantified in the task preconditions and effects.

The planning language of SIPE-2 supports definition of sort hierarchy. Some variable can be of a specific class in the sort hierarchy, for example, 'object1 of type light'. A constraint can define that some variable must not be a member of a given class, for example, 'light1 is not of type dimmable'. O-Plan2 also employs a certain level of sort hierarchy. A specification of one or more classes of objects is allowed, which are then used within a variable declaration to determine the possible set of variable bindings. For example, the classes 'types object=(b1 b2 truck plane), box=(b1 b2), vehicle=(truck plane)' could be defined to further constrain the type of the variable '?b=?{type box}'.

Preferences in plan-based HTN planners are introduced by Myers in [56, 57]. The author provided a formal language and semantics for preferences, and an algorithm implemented on top of SIPE-2. In particular, the planner accepts a preference that may condition the desired solution and the decisions steps taken during the planning process. A preference can impose prescriptions and prohibitions on characteristics of interest for some task, on the capacity in which some object is to be used in the task, and on objects. What is interesting in this approach is that there are

two forms of expressing preferences. The first form prescribes or prohibits the use of some objects when filling certain capacities in some task, while the second one prescribes or prohibits the use of a particular task when accomplishing some objective. For example, the following form expressed in natural language, “stay in a 5-star hotel while on holiday in The Netherlands”, introduces preferences on accommodation during holiday in a given country. On the other hand, the following form also expressed in natural language, “find a package bike tour starting in Groningen for the holiday in The Netherlands”, indicates that the approach taken to satisfy a particular portion of the trip should have certain characteristics, such as ‘bike’ and ‘package’, and capacity, such as ‘start location is Groningen’.

O-Plan2 allows expressing simple preferences on plans and tasks [58]. Plan preferences provide heuristic information to the planner, but it is not clear how is this information actually used, while task preferences provide information about the order in which some tasks can be decomposed, or used to handle an ‘achieve’ condition.

From a state-based HTN perspective, SHOP2 explicitly supports only hard constraints in preconditions, that is, constraints that must be satisfied by all found plans. However, there are studies that propose languages for specifying preferences and techniques that can take these constraints into account. In particular, the language and approach proposed in [59] is applied over SHOP2. The language supports three types of preferences. The first type are basic constructs of Linear Temporal Logic (LTL). The second type are soft constraints, such as the precedence constraint `before(t,t')` and the state constraints `holdBefore(t,q)`, where `t` and `t'` are tasks and `q` is a predicate. The third and most interesting type are the preferences over how tasks are decomposed into task networks (e.g., prefer to apply a certain method over another), preferences over the parametrisations of decomposed tasks (e.g., preferring one task grounding to other⁴), and a variety of temporal and non-temporal preferences over task networks. This approach is later adjusted to comply with the syntax of preferences in the version 3 of PDDL [48, 60]. Preferences over task occurrences and task decompositions are enabled by using several modal operators, which are allowed within simple and temporally extended preferences and constraints, but not within precondition preferences. A modal operator can indicate that 1) some primitive task may occur in the current state; 2) some task (or task decomposition) is initiated in the current state; or 3) some task (or task decomposition) is terminated in the current state.

SIPE-2 and O-Plan2 support our resource hierarchy completely. In addition to this hierarchy, O-Plan2 allows sharing reusable resources unary, where a sharable resource cannot be shared among many tasks at the same time, or simultaneously, where a resource can be shared among many tasks without any specific control. In SIPE-2, renewable resources can be topped up by tasks, while in O-Plan2, it can be done by tasks, some so-called off-line processes or a combination of both.

SIPE-2 offers a limited mechanism for temporal reasoning, but full support is enabled by using an external temporal reasoning system called Tachyon [61]. Tasks may include temporal constraints in three ways [35]. First, a task may implicitly define simple ordering constraints between tasks in its task network. These constraints indicate ‘before’ or ‘meets’ relationships in terms of relations defined independently by Allen [62] and van Benthem [63]. Second, explicit temporal constraints can be added to the description of a task to express any of the thirteen Allen relations between its tasks. Last, temporal constraints can be exposed on any task by using some of the temporal classes, such as ‘earliest-start’, ‘latest-end’, or ‘shortest-duration’, and some integer value to denote the time of the task. For example, if a task has assigned the expression ‘earliest-start.3’, then the “global” task network containing this task must respect the given earliest start time (that is 3) for this task.

O-Plan2 maintains a time point as a numerical pair (min,max) which denotes the upper and lower bound of any time [58]. A time point can record the start and end time of an action, the duration of an action, and delays between actions. The actions of the solution take a temporal class similar to those used by SIPE-2.

SHOP2 does not explicitly reason about time. However, there are three attempts to extend the planner to handle durative actions. The first attempt tries to produce plans with parallel actions based on concurrent update rules for numeric state variables [64]. The value of such a variable can be decreased or increased by some amount. This approach is restricted in terms of the number of relations in Allen’s algebra, and becomes inefficient when the problem size and concurrency level increases. The second approach uses a preprocessing technique that translates durative actions of level 3 in version 2.1 of PDDL [47] into SHOP2 operators to bookkeep the temporal information [13]. Parallelism is enabled only in situations where the actions begin at the same moment. The proposed algorithm is incomplete too because it avoids certain effects, axioms, and violates some ordering constraints. The last approach also enables encoding PDDL durative actions for SHOP2, where every durative action corresponds to a method composed of a ‘start’ and an ‘end’ operator [65]. The parallelism of tasks is achieved by interleaving the start and end operators. Unfortunately, this approach also shows disappointing performance when reasoning about time in large problems.

SIADEx uses three sources of temporal information [66]. The first one comes from the ordering of tasks in each task network. The planner supports three types of such ordering constraints: 1) two tasks must be executed in the total order, 2) tasks can occur in parallel, and 3) tasks may occur in any of the total orders given by their permutation. The second source is the causal structure of actions in which every action is associated with two time points, ‘start’

⁴For the sake of completeness, SHOP2 supports a similar feature called ‘sort-by’ that sorts variable bindings according to some criteria.

Table 6: Competence of HTN Planners

Property	NOAH	Nonlin	SIPE-2	O-Plan2	UMCP	SHOP2	SIADEx
Domain Dependence	Domain-configurable						
Solution Flexibility	Partial			Total			Partial
Automatic Algorithm	DFS	BFS	DFS	HS	DFS	DFS	DFS
Backtracking	×	DD	Chronological				
Heuristics	✓	✓	✓	✓	×	×	×
Search Domain-Specific Control	×	conditions	preconditions	conditions	conditions	preconditions	preconditions
Interactive Control	×	×	✓	✓	✓	×	✓
Other	CM	CM	CM	CM	CM	×	CM
Alternative Algorithms	×	×	×	×	BrFS, BFS	IDS	×
Execution Monitoring	★	×	★★★	★★★	×	×	★★★
Replanning	★	×	★★★	★★★	×	×	★★★
Completeness	×	✓	×	✓	✓	✓	
Soundness	×	×	×	×	✓	✓	

and ‘end’. Since the state is also temporally annotated, it is known which action produced some predicate and at what time it was accomplished. This information is then used to propagate constraints between tasks during the planning process and to produce flexible (sub)plan. The last source are deadline goals and complex synchronisations. The former expresses a goal that must be achieved at a particular time point, while the latter expresses constraints that allow actions to interact along a period of time.

Interpretation

From the theoretical perspective, we can conclude that HTN planning is able to express a broader and more complex set of planning problems than STRIPS-like planning. However, this statement is controversial since the assumption is that the theoretical model of HTN planning uses an infinite set of symbols to represent tasks. But in reality, this model cannot be supported by any planner unless some restrictions are imposed [44].

From the practical perspective, we may say that both categories of HTN planners are able to address similar expressiveness. It appears that planners and their corresponding category still have some challenges to address. For example, first-order logic is not fully supported by most planners, and the quality constraints are only partially implemented.

3.2.3 Competence

Table 6 summarises the properties related to the competence of HTN planners. We begin with the property of domain dependence, where all state-of-the-art HTN planners fit into the category of domain-configurable planners. The next property is on the flexibility of plans and their execution. The result of the planning process in plan-based HTN planners is a partially ordered plan which is in compliance with the definition of flexibility. An exception to this outcome is the UMCP planner which restricts the actions of the solution to be totally ordered. With respect to state-based HTN planners, there are two cases as well. SHOP2 produces a totally ordered plan, while SIADEx is able to plan for a more flexible output.

The next observation is on the search mechanism of planners. Except for Nonlin and O-Plan2, all planners implement depth-first search (DFS) as their main algorithm, however, not all of them backtrack to all alternative points. Since backtracking and decisions on variable bindings and task ordering is covered in Section 2.3.1 and Section 2.4.1, we do not go in details here. NOAH does not backtrack at all, but relies on its heuristics and choosing the “correct” task, binding or ordering among few possibilities, and ignores the rest of them. Nonlin uses breadth-first search (BrFS) with dependency-directed (DD) backtracking, that is, it backtracks on choices of variable bindings and choices of task orderings. SIPE-2 backtracks chronologically and uses heuristics to limit backtracking points to alternative tasks, allows variable binding choices only on two places during the planning process, and does not backtrack the addition of ordering constraints. O-Plan2 uses heuristic search over its choices of tasks in the plan space, where an evaluation function based on the opportunistic merit of the state is used. O-Plan2 uses an alternative manager to manage alternatives it is provided with and to seek alternatives when no results are provided by other components. It appears that both planners, O-Plan2 and SIPE-2, use resource reasoning to improve their search, but another observation says that resource reasoning is used to prune the search space in O-Plan2, while it is used to enhance the efficiency of SIPE-2 [67]. Almost all planners use their constraint management (CM) to handle constraints during the search for a solution.

Beside depth-first search, UMCP offers a choice of two more algorithms, namely breadth-first and best-first search (BFS). The planner is able to backtrack to all decision points, and along the chosen algorithm, it uses its resolution

method, and domain-specific knowledge to successfully search for a solution. SHOP2 tries all possible alternatives, either variable bindings or method choices. With respect to algorithm flexibility, the planner is provided with iterative-deepening search (IDS) as well. The authors claim that incorporation of any other search techniques is simple enough too, meaning that the structure of the planner is fairly flexible [13]. As SHOP2, SIADEx uses depth-first search too, but this planner is heavily dependent on constraint-based operations in order to successfully search for a plan.

Interaction with users throughout the planning process can be used to help the search. User interaction addresses some issues that are beyond the capabilities of the algorithms. Among HTN planners, SIPE-2, O-Plan2 and UMCP provide user interfaces for guiding purposes, while SHOP2 (in fact, its Java version) offers an interaction interface only for debugging purposes.

Backtracking points ensure completeness. Since NOAH does not backtrack to any alternative point, the planner is obviously incomplete. Nonlin is complete under the constraints defined in the domain description. Moreover, if all conditions in the domain description are set to ‘achieve’, then the planner might be complete.⁵ O-Plan2 shares the property of Nonlin, whereby the planner is complete under the requirements provided in the domain description. In other words, the planner guarantees to produce at least one valid solution if such a solution exists within the provided domain description and within the capabilities of the constraint managers. Due to extensive use of inadmissible and vaguely-defined heuristics, SIPE-2 does not guarantee that a solution will be found, although a solution may exist. This statement is even practically confirmed in [68], where the results of experiments show that SIPE-2 failed to solve feasible problems, while O-Plan2 solved all of them. Finally, UMCP and SHOP2 are provably complete and sound planners [19]. The completeness of these planners is defined with respect to both the operators, as well as the set of compound tasks.

Figure 12 shows the monitoring and replanning systems of SIPE-2, O-Plan2 and SIADEx, the only planners capable of advanced replanning and execution monitoring. The monitoring and replanning system of SIPE-2 is depicted in Figure 12a, and requires an input of arbitrary descriptions of the current plan and state in which the failure happened. This input is represented in form of predicates whose truth values should be checked. If the values of predicates differ from the expected ones, SIPE-2 uses its deductive theory to infer the changes. So far these steps are performed by the ‘Execution Monitor’. The next step is executed by the ‘Problem Recognizer’ which checks the current plan against some predefined problems that can possibly occur, and returns a list of all detected problems. Consequently, SIPE-2 through its ‘General Replanner’ tries to find so-called replanning actions that will modify the current plan. Some of these actions will insert new and unsolved goals into the plan. After having some actions found, SIPE-2 searches a solution for the modified plan which in fact represents a task network with unsolved goals. When a new solution is found, it is proceeded for execution.

The execution monitoring and replanning system of O-Plan2 is shown in Figure 12b. The input to the system consists of the current plan, some tables containing the causal structure of the plan (‘TOME’ and ‘GOST’), and some monitoring strategies. A strategy may specify to monitor all actions and report the outcome of their execution; to monitor particular actions whether they will succeed or fail during execution; to monitor specified start and end time of an action relative to a some reference point; and to report only when the whole solution has completed execution. If the information derived from the monitoring contains a failure, then there are two ways to handle that failure. First, if O-Plan2 is ignorant about the failure, it initiates a planning process with all the necessary input about the unexpected situation, and waits for a plan. Second, if O-Plan2 knows how to handle the failure (the system possesses a ‘Capability’), it tries to revise the plan, and, in case of successful repair, the system continues with execution. In case of failed attempt, the information is passed to be planned for.

SIADEx implements an execution monitoring and replanning system as shown in Figure 12c. It requires the state with exceptions and the current plan together with its casual structure as an input [69]. The monitoring process first checks whether preconditions of an action are satisfied before and whether the effects of the action are correctly applied after the action execution in the real-world state. Second, the process ensures that actions are initialised and terminated according to their temporal constraints. Furthermore, the monitoring process checks the consistency of the casual structure of the plan. If there is some link failure, this check will detect exceptions as soon as they occur. Finally, this process supports interactive suggestions, such as decisions by experts made during run time or exceptions detected by experts. The next step performed by SIADEx is repair and replanning of the exception caught. For this step, a multilevel strategy according to criticality – expresses the number of actions affected by the exception – and the nature of exception is used. The first two levels are fully autonomous and repair the plan by applying some local changes or some repair rule a priori defined. If this is not effective, a replanning is initiated to create a new plan by using the domain knowledge. The last level is the last alternative. It provides lowest degree of autonomy and it depends on an intervention of a human expert.

⁵Austin Tate, personal communication, November 23, 2012.

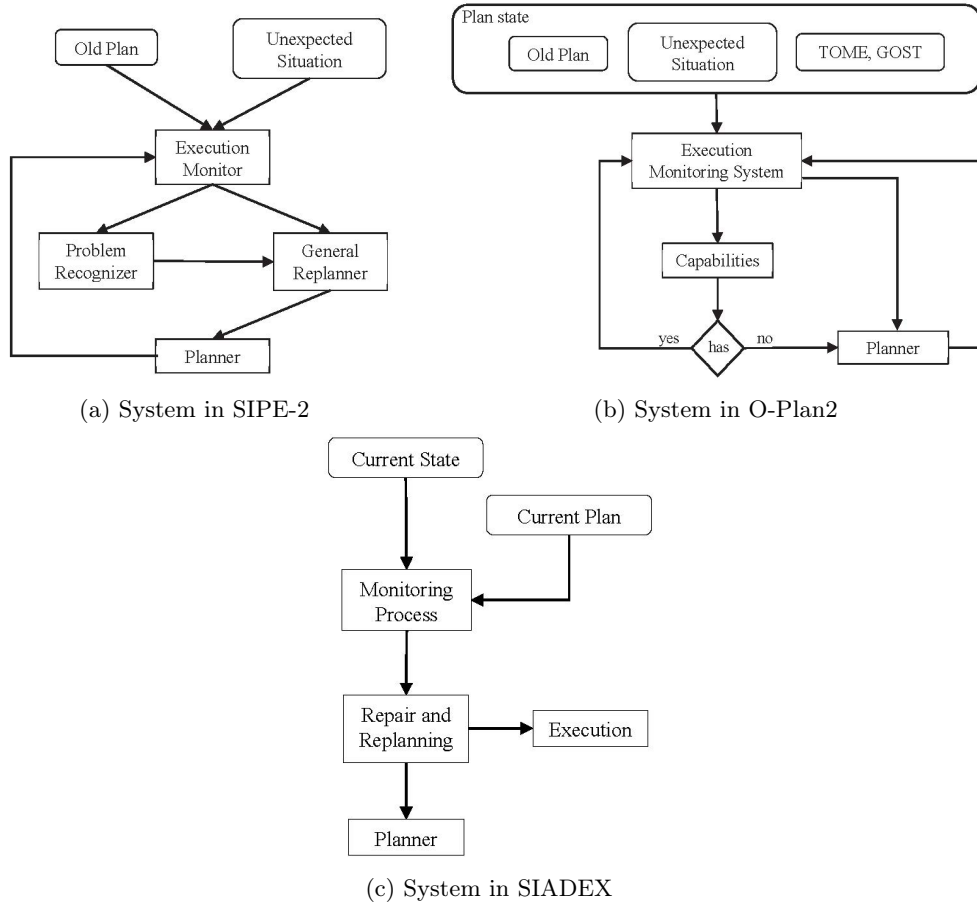


Figure 12: Execution Monitoring and Replanning in some HTN planners

Interpretation

Table 6 suggests that state-based HTN planners have much simpler search mechanisms than plan-based HTN planners. This also might confirm the statement that the underlying mechanisms of plan-based HTN are much more difficult to grasp. Furthermore, almost all planners adopt depth-first search. It is clear that the planners producing totally ordered plans are unable to support execution monitoring and replanning. With respect to execution monitoring and replanning, we provide a general architecture in Figure 13. This architecture aims to comprise functionality of HTN planners that support monitoring of plan execution and handling of failures.

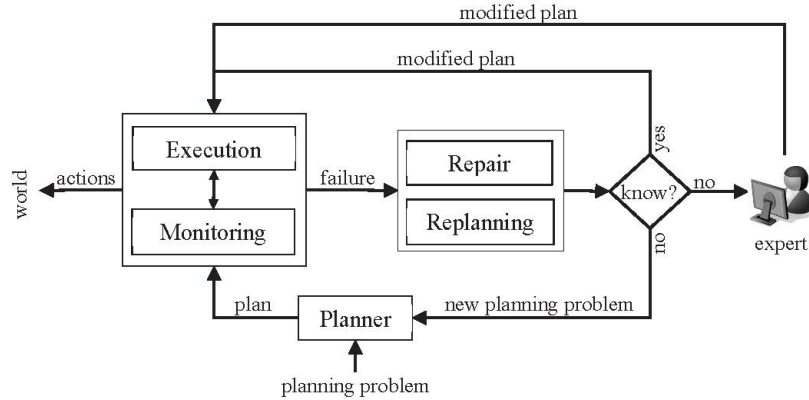


Figure 13: General Architecture for Fault Tolerance

3.2.4 Performance

Performance in Theory

The complexity results are summarised in Table 7. When no restrictions on compound tasks are imposed and task networks are partially ordered, then giving the operators and methods in the input or fixing them in advance, or allowing variables or not, does not affect the outcome and the existence of a plan is undecidable. However, given the operators and methods in the input, and being every task acyclic and every task network partially ordered, the plan existence becomes decidable.

Table 7: Time and Space Complexity of HTN Planning

Operators and methods	Compound task	Task network	Variables	Plan existence
fixed	yes	partially ordered	yes	undecidable
			no	undecidable
		totally ordered	yes	2-EXPTIME
				EXPSPACE-hard
			no	EXPTIME
				PSPACE-hard
			yes	NP-complete
			no	P
			yes	NP-complete
			no	NP-complete
input	no	partially ordered	yes	decidable
			no	decidable
		unimportant	yes	EXPSPACE-complete
			no	PSPACE-complete
				PSPACE
			yes	PSPACE-complete
			no	PSPACE-complete
			yes	PSPACE-complete
			no	PSPACE-complete
			yes	PSPACE-complete
			no	PSPACE-complete

The plan existence is decidable when task networks are totally ordered. In particular, when unrestricted compound tasks and variables are allowed, the existence is EXPSPACE-hard in double exponential time (2-EXPTIME), or, if no variable is allowed, the existence of a plan is PSPACE-hard in exponential time. When only primitive tasks and variables are allowed, the existence is NP-complete, irrespective of the ordering of task networks. Furthermore, forbidding the use of variables makes the existence to be in P. However, disallowing variables when task networks are partially ordered tasks does not change the outcome and the existence of a plan remains NP-complete.

Regardless of the ordering of task networks, when compound tasks are regular, there are two outcomes. When operators and methods are given in the input, and if variables are allowed, then the plan existence is EXPSPACE-complete, otherwise the plan existence is PSPACE-complete. When operators and methods are fixed in advance, and variables are allowed, the plan existence is PSPACE-complete in PSPACE.

Performance in Practice

Unfortunately, for most of HTN planners the performance is unknown. To the best of our knowledge, two pieces of evidence report on performance and pairwise comparison results. The first evidence compares UMCP and SHOP under loads of different problems [70]. The experiments are based on the UM Translog [71], a domain similar, but quite larger than the standard logistics domain. For this domain, a set of problems with increasing number of boxes to be delivered is randomly created. The results show that the run time for UMCP is several orders of magnitude larger than the run time for SHOP. Only in first ten problems UMCP appears to perform better than SHOP, as depicted in Table 8a. Additionally, UMCP faced some difficulties when trying to find solutions to the problems. The planner tries to solve only 37% of total number of problems, and failed 45% of those 37%. The reasons for such behaviour are due to running out of memory, inability to find an answer within a specific time frame, or returning a failure. The second evidence compares the performances of SHOP2 and SIADEx [14, 66]. The planners are tested on the Zeno travel domain [41] under a set of different temporal problems. In all cases, SIADEx outperforms the temporal version of SHOP2 [13]. Table 8b summarises the results of this comparison.

Interpretation

HTN planners, especially plan-based HTN ones, report obscure results about their performance. For example, for SIPE-2 we know that it is able to handle a domain that includes up to 200 tasks, 500 objects with 10 to 15 properties per object, and a problem that includes a few thousand predicates [40]. However, this information is insufficient to make statements about the how well SIPE-2 performs.

Table 8: Performance of some of HTN Planners

(a) Run Time and Scalability of UMCP and SHOP

Domain	Property	UMCP	SHOP
UM Translog	Run Time	10 problems > SHOP	100 problems > UMCP
	Scalability		>UMCP

(b) Run Time and Scalability of SHOP2 and SIADEx

Domain	Property	SHOP2	SIADEx
Zeno Travel	Run Time	2 problems = SIADEx	18 problems > SHOP2
	Scalability	= SIADEx	

3.2.5 Applicability

While examining the literature on HTN planners, we did our best to satisfy our curiosity about the application tendency of other than SHOP2 HTN planners which led us to Table 9 that contains a list of applications where each state-of-the-art HTN planner is applied. Particularly, O-Plan2 and SHOP2 are the most practically used planners, while SIPE-2 and SIADEx share the same and relatively high number of applications. SIPE-2 and SIADEx are applied to at least 7 domains, ranging from aircraft carrier mission [26], oncology treatment [72, 73], e-learning [74, 75] and planning tourist visits [76, 77], to a construction domain [67, 78]. SHOP2 is used to at least 11 applications, ranging from evaluation of enemy threats [79, 80], Web service composition [81, 82], to a project application [54]. Finally, O-Plan2 is applied to at least 15 domains, ranging from spacecraft mission control [83, 84], crisis management [85, 11], to a domain of biological pathway discovery [86]. Evacuation planning is tackled by O-Plan2, UMCP and SHOP2, while Web service composition is addressed also by three planners, namely O-Plan2, SHOP2 and SIADEx. From a time perspective, the majority of studies (23 in total) are performed in the decade 1990-2000, then ten studies before 2000, while in the most recent times, three applications are implemented by SIADEx.

Table 9: HTN Planners and their Applications

Application	NOAH	Nonlin	SIPE-2	O-Plan2	UMCP	SHOP2	SIADEx
Aircraft Carrier Mission Planning			[26]				
Air Campaign Planning			[87]	[88]			
Biological Pathway Discovery			[86]				
Business Process Modelling							[89]
Care Pathways							[90]
Construction Planning			[67]	[10]			
Crisis Management and Logistics				[85]			
E-learning							[74]
Electricity Turbine Overhand		[7]					
Engineering Tasks				[91]			
Equipment Configuration			[92]				
Evacuation Planning				[93]	[12]	[79]	
Evaluating Terrorist Threats						[54]	
Forest Fire Fighting						[54]	[94]
Location-Based Services						[54]	
Material Selection in Manufacturing						[54]	
Mechanical Engineering Supervision	[5]						
Military Operations Planning			[95]				
Naval Logistics		[7]					
Oil Spill Response			[96]				
Oil Tanker Truck Production				[97]			
Paediatric Oncology Treatment							[72]
Planning Tourist Visits							[76]
Production-Line Scheduling			[9]				
Project Planning				[97]		[54]	
Search and Rescue Coordination				[98]			
Software System Integration						[54]	
Spacecraft Assembly and Integration				[50]			
Spacecraft Mission Planning				[83]			
Spacecraft Platform Construction		[99]		[84]			
Statistical Goal Recognition						[54]	
Unmanned Autonomous Vehicle				[100]		[101] [102]	
US Army Small Unit Operations				[103]			
Web Service Composition				[104]		[81]	[105]
Total	1	3	7	15	1	10	7

Time Line <1980 <1990 <2000 <2010 <2020

Interpretation

Recalling our ultimate objective for AI planning, HTN planning has contributed by being employed to more than 30 applications. More than half of them are tackled with plan-based HTN planning, and O-Plan2 appears to be the most applied HTN planner, while SHOP2 is the most applied state-based HTN planner.

4 HTN Planning for Web Service Composition

Web services are software components that provide functionalities to machines that communicate over the Web. Various companies and organisations implement and offer their business logic as Web services. Adequate selection and integration of inter-organisational Web services at runtime is real challenge for modern applications on the Web. If the request of an end-user cannot be met by a single Web service, then maybe combination of already existing services will provide value-added functionality and satisfy the user request. Thus, this idea is presented exactly with the Web service composition.

Manual composition of Web services becomes impossible due to several reasons. First, the number of available Web services increases constantly which requires considerable effort to search Web service repositories for a suitable service. Second, there is no unifying description language for Web services which implies many and different description styles that affect the identification and understanding of the meaning of Web services. Third, Web services may appear or disappear dynamically and irregularly, while other may change their description over time. This dynamicity implies uncertainty about some observed state of a Web service at a particular point in time, service unavailability, and other unexpected behaviour of Web services, especially at their execution time. Finally, the user request may involve complex conditions or some policies on the behaviour of the composition as whole or only parts of it. The request may be in form of a workflow template as well. A workflow specifies activities, their ordering, the parameters for each activity, and other details used to accomplish a particular task. A workflow template is a generalisation of a workflow, in which some of the activities are declared as abstract. An abstract activity defines preferences or constraints used to match, rank and select a concrete service at run time. For example, a preference could indicate a need for services with certified authority or a need for non-free services.

These observations suggest that automated Web service composition is a challenging task. The research community of Web service composition focuses on developing tools, techniques and intelligent systems able to return a composition with the best possible quality-of-service values [106], as is the case with the Web Service Challenge [107, 108]. In the Web Service Challenge, systems, such as [109, 110, 111], generate two compositions, namely a service composition with the lowest response time, and a service composition with the highest throughput. The response time expresses the delay between the time a request is received by a Web service and the time a reply to the request is sent, and the throughput expresses the amount of requests that a Web service can handle in a given time unit. On the other hand, the AI community tries to automate the process of Web service composition by viewing the composition problem as a planning problem [112, 82, 53, 113, 114, 115, 116, 117, 118]. The general assumption is that Web service descriptions can be mapped to planning operators, that is, each service can be represented by its preconditions and effects in the planning context. Among planning techniques, HTN planning seems particularly suitable for the purpose of Web service composition because it encourages service modularity, can minimise a variety of failures or costs, scales well to large numbers of services, supports acquiring additional information by invoking Web services during planning, and enables user/software intervention [82].

Our objective is to assess how HTN planning and HTN planners address the challenges of Web service composition, and, at the same time, to assess whether these suitability statements can be confirmed.

4.1 WSC Framework

The last slice of our framework pie represents the application framework. In fact, we propose a general framework for Web service composition based on AI planning. We are interested in aspects that concern the automatic composition of Web services accomplished by planning techniques. The framework provides an abstract view without requiring a particular description language, planning algorithm, or monitoring and execution approach.

The Web service composition framework is shown in Figure 14. A Web service that models some business logic is described in some (external) language and provided to a translator that creates appropriate (internal) representation, that is, a planning problem. Consequently, the planning problem is given to a planner to search for a solution. If there is a solution found, that is, a plan, it is passed for execution and monitoring for potential failures. In case of a failure, appropriate actions are taken.

- **Service description:** The description of Web services offered to the global market usually consists of three parts. The first part refers to the information about the data transformation during the execution of a service. The

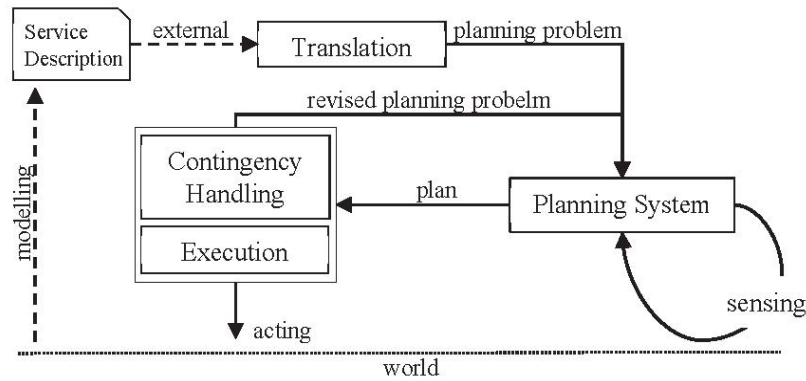


Figure 14: WSC Framework

information is presented in form of input, output and possibly exceptions. The input contains the information required for service execution, while the output presents the information the service provides after its execution. The second part refers to when and how a service transforms the world. This part consists of preconditions, that is, requirements that must be satisfied for the service to be invoked, and postconditions, that is, physical changes to be made to the world. The last part contains the non-functional properties of a service, such as cost, reliability, and service quality.

- **Translation:** Services described in a standard Web service language appear to be hard to handle by planning systems unless they are translated into an understandable form. The translation component accepts service descriptions and converts them into formal and unambiguous encoding. The result of the translation is a planning problem. In fact, this component enables the relationship between Web service composition and AI planning.
- **Planning System:** It takes the planning problem and tries to find a solution. Many planning systems distinguish between world-altering and sensing actions. The former can change the world when executed, while the latter cannot modify the state, but only acquire additional information needed to support the planning process. The most common approach is to perform off-line planning, that is, to simulate the execution of world-altering actions, and to do sensing. The solution, if it exists, consists of world-altering actions only. Many planning systems make several assumptions while planning. These assumptions simplify the planning process, but impose restrictions about what might happen in the world and distance further from the reality. The assumptions are:
 - A1:** The world is static – it can be modified only by the actions resulted from the planning process, and not by some external agent or event. All information about the world is expected to be valid till the end of the execution.
 - A2:** Sensing actions succeed – the execution of a sensing action will always return the acquired information.
 - A3:** Sensing actions are repeatable – the first sensed information is assumed to be valid for each to the same action (service) further in the planning process.
 - A4:** No changes are made to services – service’s functional properties are constant during the planning and execution process.
- **Execution Monitoring and Contingency Handling:** Considering that the world is dynamic and uncertain, the execution of actions might not proceed as expected. A contingency may be inconsistent sensed information, failures of service invocations, timeouts, or unexpected change in the world. These observations suggest that the problem of Web service composition should not be tackled decoupled from the process of action execution. Monitoring of execution and contingency handling appears to be suitable to address the aforementioned issues. Execution monitoring checks the validity of off-line calculated actions when executed and, in case of contingency, reacts appropriately. For example, if the execution time of some service takes too long, then it might be possible to proceed with the execution of subsequent actions. Other types of contingency may require repair of the existing plan, or even planning from scratch.

4.2 WSC and HTN Planning

We apply our general framework to HTN planning and approaches that employ HTN planning to compose Web services. All steps remain the same, except that some elements of the framework can be concreted now. That is, we

choose a particular service description language upon which we define the problem of Web service composition and its corresponding HTN planning problem.

OWL-S

The majority of studies that employ HTN planning and are examined here use OWL-S to describe Web services. OWL-S [119] is a Web ontology [120] for Web services used to support automated discovery, enactment and composition of Web services. The OWL-S ontology has three components: service profile, process model and service grounding. The service profile indicates the purpose of a service, and comprises the elements of part one and part three described in ‘Service description’ step in the framework. The process model indicates how to accomplish the service purpose, how to invoke the service, and what happens after the service execution. The service grounding specifies the way of interaction with the service, including a communication protocol.

OWL-S perceives services as processes. It defines three classes of processes: atomic, simple and composite. An atomic process has no sub-processes, has a grounding associated with it, and can be executed in a single step. A simple process provides an abstraction for an existing service, and has no associated grounding. A composite process consists of other processes via control constructs, such as Sequence, Split, Any-Order, etc.

WSC problem as an HTN Planning Problem

The services described in OWL-S need to be encoded in corresponding HTN elements. Intuitively, each atomic process is translated to an operator, and each simple and composite process is translated to a method. If we consider that $\mathcal{P}^W = (s_0, K, C)$ is a WSC problem described in OWL-S, where s_0 is an initial state of the world, K is a collection of OWL-S process models, and C is a composite OWL-S process defined in K , then the following relationship could be established (adopted from [121]).

Definition 15 (WSC Relationship to HTN Planning). Let $\mathcal{P}^W = (s_0, K, C)$ be an OWL-S WSC problem. Then, the sequence $p_1, \dots, p_n$, where each p_i is an atomic process defined in K is a solution to $\mathcal{P}^W$ if and only if $t_1, \dots, t_n$ is a solution to $\mathcal{P} = (Q, T_p, T_c, O, M, tn_0, s_0)$, where

- Q, T_p, T_c, O, M are generated by an OWL-S to HTN translation for the OWL-S process models K ,
- tn_0 is generated by an OWL-S to HTN translation for the OWL-S process C , and
- each t_i is a primitive task that corresponds to atomic process p_i defined by some OWL-S to HTN translation.

4.3 Review

Now that we have the Web service framework, and the relation between WSC OWL-S problem and HTN planning problem, we are able to review several studies. We group the studies into two categories. First, we analyse approaches that employ HTN planning exclusively in the attempt to solve the problem of Web service composition. Second, we discuss approaches that combine HTN planning with another technique, such as description logic and constraint satisfaction to compose services.

Approaches Based on HTN Planning Only

The earliest approach to WSC uses the predecessor of OWL-S (DAML-S [122]) to describe Web services [81]. An atomic process with effects only is translated into an operator that will simulate the effects of a world-altering Web service. An atomic process with output only is translated into an operator that has a call to a sensing Web service encoded in its precondition, and an empty delete list. A simple process is translated into a collection of methods, while a composite process is translated either into a method or a collection of methods, depending on the control construct used in the composite process. Given a composite process, a list of available processes and an incomplete initial state, the translation component creates an HTN planning problem which is in turn passed to the SHOP2 planner. Planning is performed just as described in the framework. While planning, sensing is performed with respect to the initial state meaning that the planning results would have been the same if all sensing Web services were executed first and then the planning was performed. In addition, the information provided by sensing Web services is cached in order to avoid invoking a concrete service multiple times during planning. The invocation of sensing Web services is handled by some monitor which calls an executor to execute the sensing Web service, and caches its responses. The creation of compositions is accomplished under assumptions A1-A4 which guarantee SHOP2. This approach is later adjusted to handle OWL-S [82]. In both cases, sensing Web services may return some information or may not return at all, which in turn may block the planning process. This issue is solved in [53] by augmenting SHOP2 to plan continuously

even when a sensing service has not provided any information yet. The algorithm starts with an incomplete initial state, senses needed information during planning, and explores alternative paths when some sensing Web services are executing. There is no need of explicit specification of sensing Web services in the initial description, but a query mechanism is used to search and select appropriate services when information is needed.

Madhusudan and Uttamsingh [123] suggest a system that creates compositions of Web services by using SHOP and supports execution monitoring and replanning in case of contingency. The translation between the WSC problem and HTN planning problem is not clearly defined. A catalogue of declaratively described Web services is provided. The catalogue can contain a world-altering or sensing Web service, which corresponds to an HTN operator, and a composite Web service, which corresponds to an HTN method containing a sequence of HTN operators. The goal is declaratively described, and in fact represents a goal state. Given a catalogue of Web services, methods and operators, and a goal state, SHOP uses its methods to guide the search towards reaching the goal state. Sensing is performed during planning to acquire additional information. The final plan is a sequence of world-altering actions. The execution monitoring enables consistent execution of plan steps. It updates the catalogue with the most recent information about active Web services and functional changes of Web services. In case of contingency at execution time, it is handled by executing the remaining plan in the current state, or by replanning. The choice between these two resolving options is not clearly defined. This approach is applied at least under assumptions A1 and A2.

Fernández-Olivares et al. [105] propose a middleware to interpret OWL-S descriptions by translating them into an HTN domain and problem descriptions, and to reason over those descriptions by using temporal HTN planning techniques in order to create an executable sequence of Web services. In particular, an OWL data model is translated into a PDDL data model, that is, OWL classes, properties and instances are translated into PDDL types, predicates and objects, respectively. Furthermore, an OWL-S process model is translated into an HTN domain model, that is, atomic processes are translated into PDDL durative actions, while the workflow pattern of each composite process is translated into a method associated to a compound task that corresponds to the composite process. A service request is translated into a compound task, while the OWL instance from the OWL data model represent the initial state. Such domain and problem descriptions are passed to an HTN planner. Here the SIADEx planner is employed to create a sequence of temporally annotated actions that corresponds to a composition of atomic processes. The middleware is able to monitor the execution of the composition according to the temporal information associated to the atomic processes. The execution of Web services can be performed during the planning or the plan-execution time. If a service fails or return a faulty sensed information at planning time, the planner backtracks and selects another service if possible, or tries different decomposition, if available. If some failure occurs at execution time, replanning is initiated.

The studies discussed so far use HTN planners to tackle functional properties of Web services being composed. Beside these functional properties, the service description may specify non-functional properties. These properties may include privacy, security, usability, reliability, cost, and performance, which can be used when deciding whether to include some service in a composition. Kuter and Golbeck [124] describe a taxonomy of such properties which is used to compute user's trust in a service composition process. The trust value of a service is approximated by using the ratings of users. This trust value is used to infer the trustworthiness of a composition, while the most trustworthy composition is chosen as a solution to a WSC problem. In addition to our definition of a WSC problem, here the problem has three more elements, namely the user that performs the composition, the set of all properties in the problem, and a set of user ratings. Such defined WSC problem is given as an input to an algorithm build on top of SHOP2. The assumptions made in this approach are that users' ratings are independent of each other, all services and associated ratings are given in advance, and assumption A4.

Another perspective of desired compositions is represented by different regulations required by corporations or government that provide some Web services. End-users concerned with composing such services must enforce regulations expressed as corporate policies or government rules to different aspects of compositions. Sohrabi and McIlraith [125] suggest an approach to Web service composition based on state-based HTN planning that considers such policies and regulations, while taking into account preferences over how a task can be decomposed, and preferences over services and data selection. The HTN planning problem is created from the following elements: 1) Web services are described in OWL-S and translated into HTN domain knowledge similarly as we described for the first study [82]; 2) the user's task is translated into an initial task network; 3) user preferences are represented as soft constraints in version 3 of PDDL [48], just as we described in Section 3.2.2; and 4) policies and regulations are expressed as a subset of LTL. Such planning problem is given as an input to an extended version of SHOP2 which selects services based on their functional and non-functional properties, while enforcing provided regulations. The extended planner performs best-first, incremental search and takes an HTN planning problem, a metric function and a heuristic function. The elements in the search frontier are sorted according to the heuristic function, while the metric function provides the quality value of the current plan. The planner creates a solution for the problem which represents an optimal composition for the WSC problem. The follow-up of this study enables creation of high-quality compositions through optimisation

of service and data selection [126]. The idea is to minimise the access to unknown data and to optimise efficiently data by using the independence of actions during planning. Interesting are the cases when data optimisation can be accomplished in isolation of the generation of the composition. This can happen when data is irrelevant to the optimisation of the composition (it is not mentioned in the preferences), or when the data choice does not interfere with the flow of the composition. Here atomic processes can be either world-altering or sensing. The translation of sensing atomic processes into HTN operators is modified to explicitly encode caching of sensed information. Another modification to translation is done with respect to the cases of data optimisation and associated services in such a way that the execution of a sensing service is removed and the occurrences of data are replaced with placeholders. Such sensing service is executed after the computation of the composition which suggests replacing the placeholder with an appropriate choice. While planning, the assumptions A1, A2 and A3 are made.

Approaches that Combine HTN Planning with Other Technique

The I-X planning system [127] supports services described in OWL-S and employs the O-Plan2's successor to generate Web service compositions [104]. The study does not give details on how the translation from the Web service composition problem to the planning problem is accomplished. The input to the planning system consists of a goal to be achieved, knowledge about available services and other agents it may use, and a set of some policies. Policies constrain the behaviour of the system and include procedures for, for example, digital rights management, trust management, coalition formation etc. The planning process starts by selecting an initial plan and building OWL-S profiles for all necessary services. After a number of planning steps, the system creates a plan that will achieve the goal, be consistent with these policies, and enacted by chosen services. This idea is explained on a synthetic example, called coalition search and rescue, where the request is to save or help someone in trouble by searching medical facilities and performing rescue operations according to standard policies defined in the coalition. Planning is performed under assumption A1, that is, no local threats can happen that will prevent the rescue.

Early versions of OWL-S do not provide clear specification of preconditions and effects of Web services, which consequently affects the translation from the WSC OWL-S problem into an HTN planning problem. In many cases, the translation is fixed by inserting own encodings of preconditions and effects into translated tasks. However, the expressiveness of preconditions and effects in a planner has a rather different expressiveness in OWL. Later versions of OWL-S try to solve this issue by providing a default language for specifying service preconditions and effects. In order to evaluate these preconditions and effects, planners must understand the semantics of OWL. In addition, OWL-S has open world semantics, while HTN planners assume to have complete information about the world, thus, a close-world assumption. These issues are addressed in [128] where a combination of state-based HTN planning and Description Logic (DL) for the purpose of composing Web services is investigated. Web services are described in a syntax similar to PDDL (where preconditions and effects are written in the Semantic Web Rule Language (SWRL)), while a planning problem is encoded in OWL. Such domain and problem descriptions are given as an input to an integrated system. The integration consists of an OWL reasoner and a state-based HTN planner, that is, the Pellet reasoner [129] and the Java version of SHOP planner. The planner simulates the effects of the Web services, while the reasoner takes care of the state and answers queries about the truthfulness of preconditions issued by the planner, and updates the state with simulated effects. The idea of this approach is to optimise the query answering time minimising the number of consistency checks. This approach served as a basis for the follow-up study in which compositions are created based on workflow templates [130]. As first, OWL-S is extended to support workflow templates in terms of ability to describe abstract functionalities and encode qualitative preferences. Then, since state-based HTN planning cannot directly support such extension of OWL-S, state-based HTN formalism is extended with DL knowledge about task and preference descriptions. According to the authors, the main advantage of representing services in this way is the ability to refer to profiles of the services in addition to the process descriptions. The preference descriptions about these profiles are stored in DL which means that OWL definitions can be directly used without the need for translation. Thus, the HTN-DL domain consists of a set of operators, a set of methods, and a task ontology defining the profiles and preferences related to tasks. OWL-S processes are translated into HTN elements which are handled as in usual state-based HTN algorithm. The main difference in the proposed algorithm is the matching mechanism implemented by the DL reasoner. This mechanism returns the matching actions for a given task in the order specified by the preferences. Here too, JSHOP is used as an state-based HTN planner combined with the Pellet reasoner.

Paik et al. [131, 116] combine an HTN planner with a Constraint Satisfaction Problem (CSP) solver to handle user requests that contain some additional information, such as scheduling information. A user request is translated into an HTN planning problem for which a solution containing a sequence of actions and some CSP set is computed. The CSP set corresponds to the temporal information specified in the user request. Such solution is passed to a CSP solver that calculates the final solution by binding variables with values that satisfy the temporal constraints. From an implementation perspective, JSHOP2 is used as an HTN planner, while the implementation of the CSP solver is left unspecified. From the description provided, one cannot understand how are the user requests and Web services

described, how exactly the translation between the user request and HTN planning is performed, and what assumptions are made while producing the solutions to the HTN planning and CSP problem.

Lin et al. [132] propose a composition of Web services described as OWL-S processes and preferences described as soft constraints in the version 3 of PDDL [48]. These processes and preferences are translated into an HTN planning formalism based on the definitions provided in [130] and [12]. Given an initial state, an initial task network, a set of method and operators, a task ontology, user preferences and some violation-cost function, the first step of the proposed algorithm is to preprocess the initial task network. The initial task network might be modified based on its constraints and the user preferences by adding some tasks. If there are constraints that cannot be satisfied, they are removed from the task network. After the preprocessing step, the algorithm calculates a violation-cost value for the task network which is used for selecting the possible decomposition. The value indicates the violation of some constraints (preferences) into the decomposition in the current state. Based on these costs, task networks are sorted in ascending order and the first decomposition is chosen. The algorithm is built on top of the JSHOP planner extended with the Pallet reasoner, that is, on top of the approach proposed in [130].

4.4 Summary

We propose a general and abstract framework for classifying and comparing the automatic composition of Web services based on AI planning. Furthermore, we specify some aspects of the framework when seen from the perspective of HTN planning. In particular, we choose OWL-S as a description language for Web services which enables us to provide a precise definition of the problem of Web service composition and its relationship to an HTN planning problem. Having a framework and an HTN planning problem, we review several studies that employ HTN based techniques to solve the problem. Table 10 summarises the studies with respect to indicators extracted from the framework. Some indicators are associated with rates. The rates range from '★', indicating limited focus or limited support for a respective indicator, to '★★★★', specifying comprehensive focus or extended support for the corresponding indicator. If a cell is empty, it denotes that we were not able to extract the information for the respective indicator from the literature. *Service Description* provides the language for describing Web services assumed by the study being analysed, while *translation* gives the dual information. First, it indicates how well and exactly the translation process is described, and second, which format is the Web service description translated to. *HTN Model* tells whether state-based HTN or plan-based HTN planning is employed, and which HTN planner is used for the implementation of the taken approach. Beside the extent to which it is supported, *sensing* may indicate whether the execution of a sensing action blocks the planning process, and whether sensing actions are performed during planning or they may be interleaved with world-altering ones during execution. *Assumptions* concern the degree of assumptions made to guarantee a successful composition with respect to composing, sensing and executing actions. *Contingencies* refers to unexpected behaviour of a composition at execution time, including Web service failures or time outs, and events or information changes made by some external agents. Each approach is evaluated with respect to the extent to which the support is implemented, and the type of contingency the approach can handle.

Most of the approaches assume OWL-S description of Web services, and provide sound translation algorithms to appropriate internal representation. With respect to the HTN model, all approaches but one employ state-based HTN planning. From the state-based HTN approaches, only one uses the SIADEx planner, while the rest exploit SHOP or its successor SHOP2. Most of the approaches give actual contribution to HTN planning by extending the existing algorithms or providing new algorithms on top of the existing planners. With respect to sensing, only few approaches devote appropriate attention to it and provide clear description. We can observe and conclude that sensing is done during planning and, in some cases, in a non-blocking manner. While planning, sensing and possibly executing Web services, several approaches make some of the restricting assumptions, at least those that we were able to identify from the description provided. Finally, little attention is devoted to execution monitoring and handling of contingencies at execution time.

Figure 15 gives another perspective of approaches that assume OWL-S description of Web services and provide clear translation to the planning-level representation. The lower part specifies the studies that employ HTN planning only. Sirin et al. [82] appears to be the most influential and inspiring study. Two of them are a direct extension of the study, while the other two draw inspiration from the study with respect to the translation process. The upper part depicts the studies that combine HTN planning with DL reasoning. All studies are a continuation of the work presented in the first paper on HTN planning for Web service composition [81].

5 Conclusions

An observation of studies related to HTN planning from a period of almost 40 years is likely to result in little understanding on the current theoretical, technical and practical state of the art of HTN planning. First, the field

[!ht]

Table 10: Summary of HTN-based approaches to Web service composition

Study	Web Service Description	Translation (Representation)	HTN Model (Planner)	Sensing (Properties)	Assumptions	Contingencies (Types)
[81] , [82] , [53]	DAML-S [122] OWL-S	★★★ (SHOP2)	state-based (extended SHOP2)	★★ (blocking/non-blocking, during planning)	A1-A4	× (not discussed)
[123]		★ (SHOP)	state-based (extended SHOP)	★ (interleaving)	A1,A2	★★ (replanning for failures, time outs)
[105]	OWL-S	★★ (HTN-PDDL)	state-based (SIADEx)	★★ (during planning)		★★ (replanning for failures, time outs)
[124]	OWL-S	×	state-based (extended SHOP2)	×	A4, other	×
[125] , [126]	OWL-S	★★★ (SHOP2,PDDL,LTL)	state-based (extended SHOP2)	★★ (during planning)	A1-A3	×
[104]	OWL-S		plan-based (I-X)	★ (during planning)	A1	×
[128] , [130]	OWL-S	★★★ (SHOP,DL)	state-based (JSHOP + Pellet [129])	×		×
[116]		★ (SHOP2,CSP)	state-based (JSHOP2 + CSP Solver)	×		×
[132]	OWL-S	★★★ (SHOP,DL,PDDL)	state-based (extended JSHOP + Pellet)	×		×

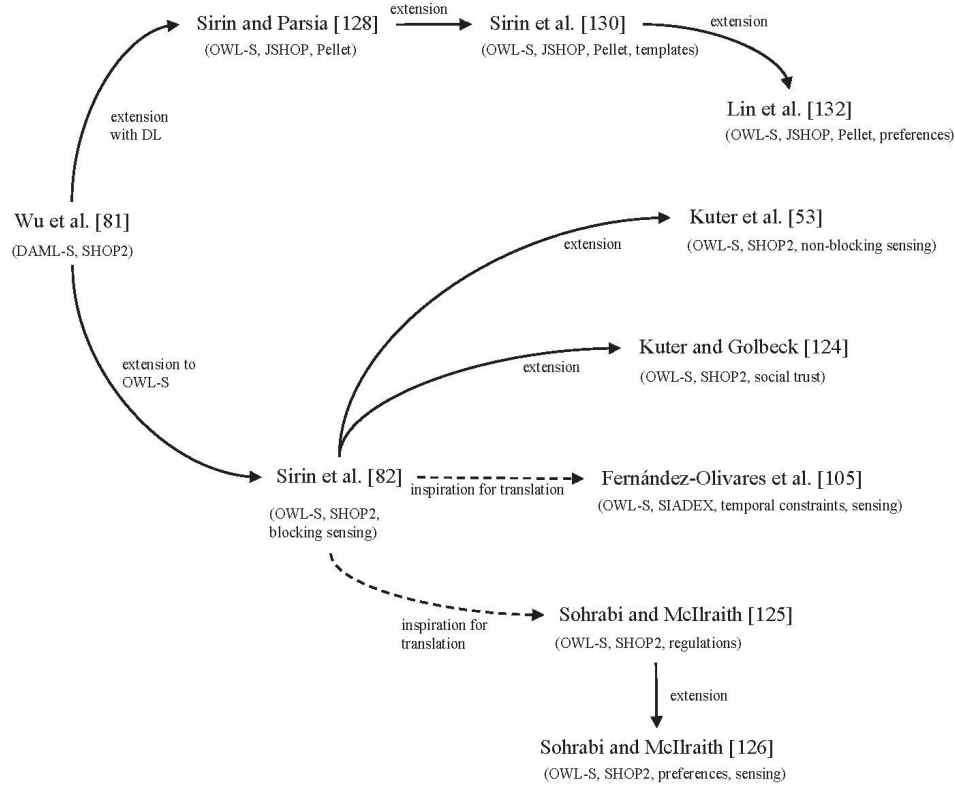


Figure 15: Relations between studies that employ HTN planning for Web service composition

encompasses studies scattered over a long period of time. Second, several HTN planners exist with interesting technical and practical implications, while only some of them received proper attention. Third, the modern and real-world applications pose challenges to planning techniques that might have not be principally envisioned and covered by existing HTN planners.

To assist in improving the outcome of such important observation, we proposed, described and evaluated a pie of frameworks whose main flavour is HTN planning. The pie is sliced into four frameworks. First, we define a theoretical framework upon which we derive two major formal models of HTN planning. Then, we design a conceptual framework which enables us to interpret differences and similarities of HTN planners from a perspective where the search space is the central concept. Third, we describe an analytical framework to examine in details several HTN planners, and to draw some conclusions more objectively. Finally, we choose Web service composition as an application domain for which we create a framework upon which we base the review and assessment of all studies that employ HTN planning for the problem of service composition.

We categorise HTN planning in plan-based HTN planning and state-based HTN planning, while we define a formal model for each category. Based on this categorisation, we group our chosen HTN planners. We find that plan-based HTN planners need to search more complex spaces than state-based HTN planners. This affects also the concepts used during the search in terms of the number of needed techniques, their technical complexity, and their interconnection. Furthermore, on an exemplifying domain, we find that plan-based HTN planners require smaller domain knowledge than state-based HTN planners. As for the expressiveness, we know that HTN planning is more expressive than STRIPS-like planning, but that none of the HTN planners can achieve that expressiveness in practice. Moreover, it appears that both categories of planners have similar levels of practical expressiveness. Almost all planners use depth-first search, but only few show flexibility for incorporating other alternative mechanisms. Based on the systems of three HTN planners, we design a general execution monitoring and replanning architecture suitable for modern planners. We had difficulties when assessing the scalability and efficiency of HTN planners as little is reported on their performance. As for the applicability of HTN planners, we find that O-Plan2 is the most applied one, followed by SHOP2. The last part of our study shows that almost all approaches to Web service composition use a pre-processing step to convert an external service representation into a possibly complete and sound internal representation suitable for HTN planners. A large majority of studies employ state-based HTN planning to solve the Web service composition problem, while the planners are able to address only partially the properties of Web service environments.

The topics explored herein suggest that there are still interesting research directions open for HTN planning. For

example, theoretical contributions in terms of new models and algorithms for HTN planning, as recently proposed search spaces and algorithms in [33]. Furthermore, a hot topic appears to be landmarks, that is, abstract tasks that occur in every solution found by an HTN planner as introduced in [133, 134]. A common syntax and semantics for specifying HTN domains and problems is missing. A description language à la PDDL can stimulate improvements of research and performance evaluation of HTN planners, can enable direct comparison of planners on possibly standardised set of problems, and finally, can help in understanding the practical expressive power of HTN planners. Another item to the list are complex goals. Rather than writing a new task or combining several tasks to accomplish a goal, or simply declaring a desired state, additional conditions may be stated in the goal, such as maintainability properties, or whether we are only interested in observing the environment or we wish to change the environment. In some situations, hybrid goals could be preferable, that is, a mixture of declarative and imperative representation of the objective, similarly to [135]. Web service composition indicates that planning under uncertainty is an inevitable necessity for most real-world environments. For example, almost every approach we reviewed executes sensing services during planning only (while making several restricting assumptions). In this context, there are few approaches that augment HTN planning to cope with incomplete knowledge, such as [136]. Planning under uncertainty also considers non-determinism of actions, meaning that actions may have different possible outcomes. The issue of non-determinism is discussed in [137], which may serve as initial point for further exploration. Having incomplete knowledge and actions that provide information, it is highly possible that an HTN planner would have to confront different types of failures and action unavailability. An appropriate reaction to failures, time-outs, external events or changes made by some agents that are beyond the control of the planner is still a real challenge for HTN planners. Continual planning [138] investigates these problems and stimulates active sensing. Furthermore, a non-classical domain may require to distribute planning over different agents present in the surroundings. In this context, only few studies investigate planning and distribution, both, in plan-based HTN planning, such as distributed NOAH [139] and distributed SIPE-2 [140], and state-based HTN planning, such as A-SHOP [141, 142], and Planner9 [143], while most of them provide minor theoretical and practical contributions. In addition, although Web service composition is a real-world challenge, we remark that there are no results on evaluation of the proposed approaches in a real setting. The approaches are tested on simple scenarios, examples or even classical problems that do not necessary lead to conclude that a particular approach will be equally successful or unfavourable in a real setting. Finally, it may seem far from the purpose of automated planning, but creating interfaces that are user-friendly and intuitive could encourage the end-user to consider even more HTN planning for solving real-world problems. Instead of configuring files and taking low-level steps to invoke a planner, the users could interact with a graphical interface which would enable them to model a domain, create and store a goal, follow the planning process, or examine the final solution. SIPE-2 and O-Plan2 are good examples of HTN planners enhanced with components for human-computer interaction.

Acknowledgement

We are grateful to Luigia Carlucci Aiello and Eirini Kaldeli for the helpful comments on the paper. We would like to thank Austin Tate for valuable comments on the Nonlin and O-Plan2 planning systems and Luis Castillo Vidal for the valuable feedback on the SIADEx planning system. The work is supported by the Dutch National Research Council under the NWO Smart Energy Systems program, contract no. 647.000.004.

References

- [1] M. Ghallab, D. S. Nau, and P. Traverso, *Automated Planning: Theory & Practice*. Morgan Kaufmann Publishers Inc., 2004.
- [2] D. S. Nau, Y. Cao, A. Lotem, and H. Muñoz Avila, “SHOP: Simple Hierarchical Ordered Planner,” in *Proceedings of the 16th International Joint Conference on Artificial Intelligence*, ser. IJCAI’99. Morgan Kaufmann Publishers Inc., 1999, pp. 968–975.
- [3] F. Bacchus, “The AIPS ’00 Planning Competition,” *AI Magazine*, vol. 22, no. 3, pp. 47–56, 2001.
- [4] D. S. Nau, “Current Trends in Automated Planning,” *AI Magazine*, vol. 28, no. 4, pp. 43–58, 2007.
- [5] E. D. Sacerdoti, “A Structure for Plans and Behavior,” Ph.D. dissertation, Standfor University, AI Center, 1975, aAI7605794.
- [6] E. D. Sacerdoti, “The Nonlinear Nature of Plans,” in *Proceedings of the 4th international Joint Conference on Artificial Intelligence - Volume 1*, ser. IJCAI’75. Morgan Kaufmann Publishers Inc., 1975, pp. 206–214.
- [7] A. Tate, “Project Planning Using a Hierarchic Non-linear Planner,” Department of Artificial Intelligence, University of Edinburgh, Research Report 25, aug 1976.
- [8] A. Tate, “Generating Project Networks,” in *Proceedings of the 5th International Joint Conference on Artificial Intelligence - Volume 2*, ser. IJCAI’77. Morgan Kaufmann Publishers Inc., 1977, pp. 888–893.

- [9] D. E. Wilkins, “Can AI Planners Solve Practical Problems?” *Comput. Intell.*, vol. 6, pp. 232–246, January 1991.
- [10] K. Currie and A. Tate, “O-Plan: The Open Planning Architecture,” *Artificial Intelligence*, vol. 52, no. 1, pp. 49–86, Nov. 1991.
- [11] A. Tate, B. Drabble, and R. Kirby, “O-Plan2: An Open Architecture for Command, Planning and Control,” in *Intelligent Scheduling*. Morgan Kaufmann Publishers Inc., 1994, pp. 213–239.
- [12] K. Erol, “Hierarchical Task Network Planning: Formalization, Analysis, and Implementation,” Ph.D. dissertation, Computer Science Department, University of Maryland, 1996.
- [13] D. S. Nau, O. Ilghami, U. Kuter, J. W. Murdock, D. Wu, and F. Yaman, “SHOP2: An HTN Planning System,” *Journal of Artificial Intelligence Research*, vol. 20, no. 1, pp. 379–404, Dec. 2003.
- [14] L. A. Castillo, J. Fernández-Olivares, Ó. García-Pérez, and F. Palao, “Temporal Enhancements of an HTN Planner,” in *Proceedings of the 11th Conference of the Spanish Association for Artificial Intelligence, Current Topics in AI*. Springer-Verlag, 2005, pp. 429–438.
- [15] M. P. Papazoglou and D. Georgakopoulos, “Introduction: Service-Oriented Computing,” *Comm. ACM*, vol. 46, no. 10, pp. 24–28, oct 2003.
- [16] A. Lazovik, M. Aiello, and M. Papazoglou, “Associating Assertions with Business Processes and Monitoring Their Execution,” in *Proceedings of the 2nd international conference on Service oriented computing*, ser. ICSOC ’04. ACM, 2004, pp. 94–104.
- [17] S. Dustdar and W. Schreiner, “A Survey on Web Services Composition,” *Int. J. Web Grid Serv.*, vol. 1, no. 1, pp. 1–30, aug 2005.
- [18] E. Kaldeli, “Domain-Independent Planning for Services in Uncertain and Dynamic Environments,” Ph.D. dissertation, Faculty of Mathematics and Natural Sciences, University of Groningen, 2013.
- [19] K. Erol, J. Hendler, and D. S. Nau, “UMCP: A Sound and Complete Procedure for Hierarchical Task-Network Planning,” in *International Conference on AI Planning Systems*. Morgan Kaufmann Publishers Inc., 1994, pp. 249–254.
- [20] T. Geier and P. Bercher, “On the Decidability of HTN Planning with Task Insertion,” in *Proceedings of the Twenty-Second international joint conference on Artificial Intelligence - Volume Volume Three*, ser. IJCAI’11. AAAI Press, 2011, pp. 1955–1961.
- [21] M. Stefik, “Planning with Constraints (MOLGEN: Part 1),” *Artificial Intelligence*, vol. 16, no. 2, pp. 111–140, 1981.
- [22] D. S. Weld, “An Introduction to Least Commitment Planning,” *AI Magazine*, vol. 15, no. 4, pp. 27–61, 1994.
- [23] R. Tsuneto, K. Erol, J. Hendler, and D. S. Nau, “Commitment Strategies in Hierarchical Task Network Planning,” in *Proceedings of the Thirteenth National Conference on Artificial Intelligence - Volume 1*, ser. AAAI’96. AAAI, 1996, pp. 536–542.
- [24] D. E. Foulser, M. Li, and Q. Yang, “Theory and Algorithms for Plan Merging,” *Artificial Intelligence*, vol. 57, pp. 143–181, October 1992.
- [25] Q. Yang, “A Theory of Conflict Resolution in Planning,” *Artificial Intelligence*, vol. 58, no. 1, pp. 361–392, 1992.
- [26] D. E. Wilkins, *Practical Planning: Extending the Classical AI Planning Paradigm*. Morgan Kaufmann Publishers Inc., 1988.
- [27] K. Erol, J. Hendler, and D. S. Nau, “Semantics for Hierarchical Task-Network Planning,” University of Maryland, Institute for Advanced Computer Studies, Tech. Rep. UMIACS-TR-94-31, 1994.
- [28] A. Nareyek, E. C. Freuder, R. Fourer, E. Giunchiglia, R. P. Goldman, H. Kautz, J. Rintanen, and A. Tate, “Constraints and AI Planning,” *IEEE Intelligent Systems*, vol. 20, pp. 62–72, March 2005.
- [29] I. Georgievski, A. Lazovik, and M. Aiello, “Task Interaction in an HTN Planner,” CoRR, Tech. Rep., 2011.
- [30] R. E. Fikes and N. J. Nilsson, “STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving,” in *Proceedings of the 2nd International Joint Conference on Artificial Intelligence*, ser. IJCAI’71. Morgan Kaufmann Publishers Inc., 1971, pp. 608–620.
- [31] S. Kambhampati, “A Comparative Analysis of Partial Order Planning and Task Reduction Planning,” *SIGART Bull.*, vol. 6, pp. 16–25, January 1995.
- [32] G. Collins and L. Pryor, “Achieving the Functionality of Filter Conditions in a Partial-Order Planner,” in *Proceedings of the 10th National Conference on Artificial Intelligence*, ser. AAAI’92. AAAI, 1992, pp. 375–380.
- [33] R. Alford, V. Shivashankar, U. Kuter, and D. S. Nau, “HTN Problem Spaces: Structure, Algorithms, Termination,” in *Proceedings of the Fifth Annual Symposium on Combinatorial Search*. AAAI Press, 2012, pp. 2–9.
- [34] D. Chapman, “Planning for Conjunctive Goals,” *Artificial Intelligence*, vol. 32, no. 3, pp. 333–377, Jul. 1987.
- [35] D. E. Wilkins, “Using the SIPE-2 Planning System - A Manual for SIPE-2,” 2000.
- [36] A. Tate, B. Drabble, and J. Dalton, “The Use of Condition Types to Restrict Search in an AI Planner,” in *Proceedings of the Twelfth National Conference on Artificial Intelligence - Volume 2*, ser. AAAI’94. AAAI, 1994, pp. 1129–1134.

- [37] R. Tsuneto, J. Hendler, and D. S. Nau, “Analyzing External Conditions to Improve the Efficiency of HTN Planning,” in *Proceedings of the 15th National/10th Conference on Artificial Intelligence/Innovative Applications of Artificial Intelligence*. AAAI, 1998, pp. 913–920.
- [38] “SHOP, SHOP2, JSHOP and JSHOP-2 Planners,” Online: accessed Jan. 2014, <http://sourceforge.net/projects/shop/>.
- [39] “Python 3.4.0,” Online: accessed Jan. 2014, <http://www.python.org/getit/releases/3.4.0/>.
- [40] D. E. Wilkins and M. Desjardins, “A Call for Knowledge-Based Planning,” *AI Magazine*, vol. 22, no. 1, pp. 99–115, 2001.
- [41] D. Long and M. Fox, “The 3rd International Planning Competition: Results and Analysis,” *J. Artif. Intell. Res. (JAIR)*, vol. 20, pp. 1–59, 2003.
- [42] K. Erol, J. A. Hendler, and D. S. Nau, “HTN Planning: Complexity and Expressivity,” in *Proceedings of the 12th National Conference on Artificial Intelligence - Volume 2*, ser. AAAI’94. AAAI, 1994, pp. 1123–1128.
- [43] D. S. Nau, S. J. Smith, and K. Erol, “Control Strategies in HTN Planning: Theory Versus Practice,” in *Proceedings of the 15th National/10th conference on Artificial Intelligence/Innovative applications of Artificial Intelligence*, ser. AAAI’98/IAAI’98. AAAI, 1998, pp. 1127–1133.
- [44] M. Lekavý and P. Návrát, “Expressivity of STRIPS-Like and HTN-Like Planning,” in *Proceedings of the 1st KES International Symposium on Agent and Multi-Agent Systems: Technologies and Applications*, ser. KES-AMSTA’07. Springer-Verlag, 2007, pp. 121–130.
- [45] K. Erol, J. Handler, and D. S. Nau, “Complexity Results for HTN Planning,” *Annals of Mathematics and AI*, vol. 18, no. 1, pp. 69–93, 1996.
- [46] M. Ghallab, C. K. Isi, S. Penberthy, D. E. Smith, Y. Sun, and D. Weld, “PDDL - The Planning Domain Definition Language,” CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, Tech. Rep., 1998.
- [47] M. Fox and D. Long, “PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains,” *Journal of Artificial Intelligence Research*, vol. 20, no. 1, pp. 61–124, Dec. 2003.
- [48] A. Gerevini and D. Long, “Preferences and Soft Constraints in PDDL3,” in *Proceedings of the ICAPS Workshop on Planning with Preferences and Soft Constraints*, 2006.
- [49] J. M. Agosta, “Constraining Influence Diagram Structure by Generative Planning: An Application to the Optimization of Oil Spill Response,” in *Proceedings of the Twelfth international Conference on Uncertainty in Artificial Intelligence*, ser. UAI’96. Morgan Kaufmann Publishers Inc., 1996, pp. 11–19.
- [50] M. Aarup, M. M. Arentoft, Y. Parrod, I. Stokes, H. Vadon, and J. Stader, “OPTIMUM-AIV: A Knowledge-Based Planning and Scheduling System for Spacecraft AIV,” in *Proceedings of the Conference on Knowledge Based Scheduling*. Morgan Kaufmann Publishers Inc., 1994, pp. 451–469.
- [51] S. J. J. Smith, K. Hebbbar, D. S. Nau, and I. Minis, “Integrating Electrical and Mechanical Design and Process Planning,” 1997.
- [52] E. Kaldeli, E. U. Warriach, A. Lazovik, and M. Aiello, “Coordinating the Web of Services for a Smart Home,” *ACM Transactions on the Web*, vol. 7, no. 2, pp. 10:1–10:40, 2012.
- [53] U. Kuter, E. Sirin, B. Parsia, D. Nau, and J. Hendler, “Information Gathering during Planning for Web Service Composition,” *Web Semantic*, vol. 3, pp. 183–205, October 2005.
- [54] D. S. Nau, T. C. Au, O. Ilghami, U. Kuter, D. Wu, F. Yaman, H. Muñoz Avila, and J. W. Murdock, “Applications of SHOP and SHOP2,” *IEEE Intelligent Systems*, vol. 20, no. 2, pp. 34–41, march-april 2005.
- [55] B. Schattenberg, “Hybrid planning and scheduling,” Ph.D. dissertation, Institute of Artificial Intelligence, Ulm University, 2009.
- [56] K. L. Myers, “Strategic Advice for Hierarchical Planners,” in *Proceedings of the 5th International Conference on Principles of Knowledge Representation and Reasoning*. Morgan Kaufmann Publishers Inc., 1996, pp. 112–123.
- [57] K. L. Myers, “Planning with Conflicting Advice,” in *Proceedings of the 5th International Conference on Artificial Intelligence Planning Systems*. AAAI, 2000, pp. 355–362, poster Paper.
- [58] B. Drabble and A. Tate, “O-Plan: Task Formalism Manual Version 2.3,” AIAI, University of Edinburgh, Tech. Rep., jul 1995.
- [59] S. Sohrabi and S. A. McIlraith, “On Planning with Preferences in HTN,” in *Proceedings of the 12th International Workshop on Non-Monotonic Reasoning*, 2008, pp. 241–248.
- [60] S. Sohrabi, J. A. Baier, and S. A. McIlraith, “HTN Planning with Preferences,” in *Proceedings of the 21st International Joint Conference on Artificial Intelligence*, ser. IJCAI’09. Morgan Kaufmann Publishers Inc., 2009, pp. 1790–1797.
- [61] J. Stillman, R. Arthur, and A. Deitsch, “Tachyon: A Constraint-based Temporal Reasoning Model and Its Implementation,” *SIGART Bull.*, vol. 4, no. 3, pp. 1–4, jul 1993.
- [62] J. F. Allen, “Maintaining Knowledge about Temporal Intervals,” *Comm. ACM*, vol. 26, no. 11, pp. 832–843, 1983.

- [63] J. F. A. K. v. Benthem, *The Logic of Time : A Model-Theoretic Investigation into the Varieties of Temporal Ontology and Temporal Discourse*, 2nd ed. Kluwer Academic Publishers Dordrecht, 1991.
- [64] F. Yaman and D. S. Nau, "Timeline: An HTN Planner that can Reason about Time," in *Proceedings of the AIPS'02 Workshop on Planning for Temporal Domains*, 2002, pp. 75–81.
- [65] R. P. Goldman, "Durative Planning in HTNs," in *Proceedings of the 16th International Conference on Automated Planning and Scheduling*, ser. ICAPS'06. AAAI, 2006, pp. 382–385.
- [66] L. A. Castillo, J. Fernández-Olivares, Ó. García-Pérez, and F. Palao, "Efficiently Handling Temporal Knowledge in an HTN Planner," in *Proceedings of the 16th International Conference on Automated Planning and Scheduling*, ser. ICAPS'06. AAAI, 2006, pp. 63–72.
- [67] N. A. Kartam and D. E. Wilkins, "Toward a Foundation for Evaluating AI Planners," AI Center, SRI International, Tech. Rep., 1989.
- [68] C. Gomes, "O-plan2 vs sipe-2 - a general comparison," Rome Laboratory, Tech. Rep., jul 1996.
- [69] J. Fernández-Olivares, I. Sánchez-Garzón, A. González-Ferrer, J. A. Cózar, A. Fdez-Tejjeiro, M. R. Cabello, and L. Castillo, "Task Network Based Modeling, Dynamic Generation and Adaptive Execution of Patient-tailored Treatment Plans Based on Smart Process Management Technologies," in *Proceedings of the 3rd International Conference on Knowledge Representation for Health-Care*, ser. KR4HC'11. Springer-Verlag, 2012, pp. 37–50.
- [70] D. S. Nau, Y. Cao, A. Lotem, and H. Muñoz Avila, "SHOP and M-SHOP: Planning with Ordered Task Decomposition," Computer Science Department, University of Maryland, Tech. Rep. CS-TR-4157, jun 2000.
- [71] S. Andrews, B. Kettler, K. Erol, and J. Hendler, "UM Translog: A Planning Domain for the Development and Benchmarking of Planning Systems," Computer Science Department, University of Maryland, Tech. Rep. CS-TR-3487, 1995.
- [72] J. Fernández-Olivares, J. A. Cózar, and L. Castillo, "Automating Oncology Therapy Plans by means of Temporal Hierarchical Task Networks Planning," in *Proceedings of the ECAI Workshop on Knowledge Management for Healthcare Processes*, 2008.
- [73] J. Fdez-Olivares, L. Castillo, J. A. Cózar, and O. García Pérez, "Supporting Clinical Processes and Decisions by Hierarchical Planning and Scheduling," *Computational Intelligence*, vol. 27, no. 1, pp. 103–122, 2011.
- [74] L. Castillo, L. Morales, A. González-Ferrer, J. Fernández-Olivares, D. Borrajo, and E. Onaindía, "Automatic Generation of Temporal Planning Domains for e-Learning Problems," *J. of Scheduling*, vol. 13, no. 4, pp. 347–362, aug 2010.
- [75] A. Garrido, S. Fernández, L. Morales, E. Onaindía, D. Borrajo, and L. Castillo, "On the Automatic Compilation of e-Learning Models to Planning," *Knowledge Eng. Review*, vol. 28, no. 2, pp. 121–136, 2013.
- [76] L. Castillo, E. Armengol, E. Onaindía, L. Sebastián, J. González-Boticario, A. Rodríguez, S. Fernández, J. D. Arias, and D. Borrajo, "Samap: An User-oriented Adaptive System for Planning Tourist Visits," *Expert Syst. Appl.*, vol. 34, no. 2, pp. 1318–1332, 2008.
- [77] F. Palao, L. Castillo, J. Fernández-Olivares, and O. García, "Cities That Offer a Customized and Personalized Tourist Experience to Each and Every Visitor: The Smartourism Project," in *Proceedings of the AI for an Intelligent Planet*. ACM, 2011, pp. 4:1–4:8.
- [78] N. A. Kartam, R. E. Levitt, and D. E. Wilkins, "A Centralized Approach for Representing and Resolving Interactions among Multi-Agent Tasks while Planning Hierarchically," in *Proceedings of the 6th Conference on Artificial Intelligence Applications*. IEEE Press, 1990, pp. 250–256.
- [79] H. Muñoz-Avila, D. W. Aha, L. Breslow, and D. S. Nau, "HICAP: An Interactive Case-Based Planning Architecture and its Application to Noncombatant Evacuation Operations," 1999.
- [80] H. Muñoz-Avila, D. W. Aha, D. S. Nau, R. Weber, L. Breslow, and F. Yamal, "SiN: Integrating Case-Based Reasoning with Task Decomposition," in *Proceedings of the 17th International Joint Conference on Artificial Intelligence - Volume 2*, ser. IJCAI'01. Morgan Kaufmann Publishers Inc., 2001, pp. 999–1004.
- [81] D. Wu, B. Parsia, E. Sirin, J. Hendler, and D. S. Nau, "Automating DAML-S Web Services Composition Using SHOP2," in *Proceedings of 2nd International Semantic Web Conference*, ser. ISWC'03. Springer-Verlag, 2003, pp. 195–210.
- [82] E. Sirin, B. Parsia, D. Wu, J. Hendler, and D. S. Nau, "HTN Planning for Web Service Composition Using SHOP2," *Web Semantic*, vol. 1, pp. 377–396, October 2004.
- [83] M. E. Drummond, K. W. Currie, and A. Tate, "O-Plan Meets T-SAT: First Results From the Application of an AI Planner to Spacecraft Mission Sequencing," Artificial Intelligence Applications Institute, University of Edinburgh, Tech. Rep., 1988.
- [84] B. Drabble, J. Dalton, and A. Tate, "Repairing Plans On-the-fly," in *Proceedings of the NASA Workshop on P&S for Space*, 1997.
- [85] A. Tate, B. Drabble, and J. Dalton, "O-Plan: A Knowledge-Based Planner and its Application to Logistics," in *Proceedings of ARPI*, 1996, pp. 259–266.

- [86] S. Khan, W. Gillis, C. Schmidt, and K. Decker, "A Multi-Agent System-driven AI Planning Approach to Biological Pathway Discovery," in *Proceedings of the 13th International Conference on Automated Planning and Scheduling*, ser. ICAPS'03. AAAI, 2003, pp. 246–255.
- [87] T. J. Lee and D. Wilkins, "Using SIPE-2 to Integrate Planning for Military Air Campaigns," *IEEE Expert*, vol. 11, no. 6, pp. 11–12, Dec 1996.
- [88] A. Tate, J. Dalton, and J. Levine, "Generation of Multiple Qualitatively Different Plan Options," in *Proceedings of the 4th International Conference on AI Planning Systems*. AAAI, 1998, pp. 27–35.
- [89] A. González-Ferrer, J. Fernández-Olivares, and L. Castillo, "From Business Process Models to Hierarchical Task Network Planning Domains," *Knowledge Eng. Review*, vol. 28, no. 2, pp. 175–193, 2013.
- [90] I. Sánchez-Garzón, J. Fernández-Olivares, and L. Castillo, "An Approach for Representing and Managing Medical Exceptions in Care Pathways Based on Temporal Hierarchical Planning Techniques," in *Process Support and Knowledge Representation in Health Care*, ser. Lecture Notes in Computer Science, R. Lenz, S. Miksch, M. Peleg, M. Reichert, D. Riano, and A. Teije, Eds. Springer, 2013, vol. 7738, pp. 168–182.
- [91] A. Tate, "Responsive Planning and Scheduling using AI Planning Techniques - Optimum-AIV," *IEEE Expert: Intelligent Systems & their Applications*, vol. 11, no. 6, pp. 4–12, 1996.
- [92] J. M. Agosta, "Formulation and Implementation of an Equipment Configuration Problem with the (SIPE-2) Generative Planner," in *Proceedings of the AAAI-95 Spring Symposium on Integrated Planning Applications*, 1995, pp. 1–10.
- [93] A. Tate, J. Dalton, and J. Levine, "O-Plan: A Web-Based AI Planning Agent," in *Proceedings of the 17th National Conference on Artificial Intelligence and 12th Conference on Innovative Applications of Artificial Intelligence*, ser. AAAI'00/IAAI'00. AAAI, 2000, pp. 1131–1132.
- [94] M. de la Asunción, L. Castillo, J. Fernández-Olivares, O. García-Pérez, A. González, and F. Palao, "SIADDEX: An Interactive Artificial Intelligence Planner for Decision Support and Training in Forest Fire Fighting," *Artificial Intelligence Communications*, vol. 18, no. 4, pp. 257–268, 2005.
- [95] D. Wilkins and R. V. Desimone, "Applying an AI Planner to Military Operations Planning," in *Intelligent Scheduling*. Morgan Kaufmann Publishers Inc., 1992, pp. 685–709.
- [96] "Oil Spill Response Planning in SIPE-2," Online: accessed Jan. 2014, <http://www.ai.sri.com/sipe/oil.html>.
- [97] A. Directorate, "Alvey Grand Meeting of Community Clubs," Available through Institution of Electrical Engineering, Tech. Rep., 1987.
- [98] J. Kingston, N. Shadbolt, and A. Tate, "CommonKADS Models for Knowledge Based Planning," in *Proceedings of the National Conference on Artificial Intelligence*, ser. AAAI'96. AAAI, 1996, pp. 11–6.
- [99] A. Tate, *Interfacing a CAD System to an AI Planner*, ser. AIAI-TR-76. Artificial Intelligence Applications Institute, Univ. of Edinburgh, 1990.
- [100] A. Tate and J. Dalton, "O-Plan: a Common Lisp Planning Web Service," in *Proceedings of the International Lisp Conference*, 2003.
- [101] "Real-time coordination and control of multiple heterogeneous UAV," Online: accessed Jan. 2014, <http://www.comets-uavs.org/>.
- [102] "Smart Information Flow Technologies - Planning," Online: accessed Jan. 2014, <http://www.sift.net/projects/planning>.
- [103] A. Tate, J. Levine, P. Jarvis, and J. Dalton, "Using AI Planning Technology for Army Small Unit Operations," in *Proceedings of the Artificial Intelligence Planning and Scheduling Systems Conference*. AAAI, 2000, pp. 379–386, poster Paper.
- [104] A. Uszok, J. M. Bradshaw, R. Jeffers, A. Tate, and J. Dalton, "Applying KAoS Services to Ensure Policy Compliance for Semantic Web Services Workflow Composition and Enactment," in *International Semantic Web Conference*. Springer-Verlag, 2004, pp. 425–440.
- [105] J. Fernández-Olivares, T. Garzón, L. Castillo, O. García-Pérez, and F. Palao, "A Middle-Ware for the Automated Composition and Invocation of Semantic Web Services Based on Temporal HTN Planning Techniques," in *Current Topics in Artificial Intelligence*, ser. Lecture Notes in Computer Science, D. Borrajo, L. Castillo, and J. Corchado, Eds. Springer, 2007, vol. 4788, pp. 70–79.
- [106] S. Bleul, T. Weise, and K. Geihs, "The Web Service Challenge – A Review on Semantic Web Service Composition," in *Service-Oriented Computing (SOC'2009)*, Mar 2009, pp. 1–12.
- [107] "The Web Services Challenge," Online: accessed Jan. 2014, <http://www.wschallenge.org/>.
- [108] M. B. Blake, W. Cheung, M. C. Jaeger, and A. Wombacher, "WSC-06: The Web Service Challenge," in *Joint Proceedings of the CEC/EEE 2006*, 2006, pp. 1–2.
- [109] M. Aiello, N. van Benthem, and E. el Khoury, "Visualizing Compositions of Services from Large Repositories," in *Joint Proceedings of the IEEE CEC/EEE 2008*, 2008, pp. 359–362.

- [110] M. Aiello, E. el Khoury, A. Lazovik, and P. Ratelband, "Optimal QoS-Aware Web Service Composition," in *Joint Proceedings of the IEEE CEC/EEE 2009*, 2009, pp. 491–494.
- [111] V. Degeler, I. Georgievski, A. Lazovik, and M. Aiello, "Concept Mapping for Faster QoS-Aware Web Service Composition," in *IEEE Conference on Service Oriented Computing and Applications*, 2010, pp. 1–4.
- [112] M. Aiello, M. P. Papazoglou, J. Yang, M. Carman, M. Pistore, L. Serafini, and P. Traverso, "A Request Language for Web-Services Based on Planning and Constraint Satisfaction," in *Proceedings of the Third International Workshop on Technologies for E-Services*. Springer-Verlag, 2002, pp. 76–85.
- [113] B. Medjahed and A. Bouguettaya, "A Multilevel Composability Model for Semantic Web Services," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, pp. 954–968, July 2005.
- [114] M. Klusch and A. Gerber, "Semantic Web Service Composition Planning with OWLS-XPlan," in *Proceedings of the 1st International AAAI Fall Symposium on Agents and the Semantic Web*. AAAI, 2005, pp. 55–62.
- [115] S. Sohrabi, N. Prokoshyna, and S. A. McIlraith, "Web Service Composition via Generic Procedures and Customizing User Preferences," in *Proceedings of the International Semantic Web Conference*, ser. ISWC'06. Springer-Verlag, 2006, pp. 597–611.
- [116] I. Paik and D. Maruyama, "Automatic Web Services Composition Using Combining HTN and CSP," in *Proceedings of the 7th IEEE International Conference on Computer and Information Technology*. IEEE Computer Society, 2007, pp. 206–211.
- [117] E. Kaldeli, A. Lazovik, and M. Aiello, "Extended Goals for Composing Services," in *Proceedings of the 19th International Conference on Automated Planning and Scheduling*, ser. ICAPS'09. AAAI, 2009, pp. 362–365.
- [118] E. Kaldeli, A. Lazovik, and M. Aiello, "Continual Planning with Sensing for Web Service Composition," in *Proceedings of the 25th AAAI Conference on Artificial Intelligence*. AAAI, 2011, pp. 1198–1203.
- [119] D. Martin, M. Burstein, D. McDermott, S. McIlraith, M. Paolucci, K. Sycara, D. L. McGuinness, E. Sirin, and N. Srinivasan, "Bringing Semantics to Web Services with OWL-S," *World Wide Web*, vol. 10, no. 3, pp. 243–277, 2007.
- [120] I. Horrocks, P. F. Patel-Schneider, and F. van Harmelen, "From {SHIQ} and {RDF} to OWL: the Making of a Web Ontology Language," *Web Semantics: Science, Services and Agents on the World Wide Web*, vol. 1, no. 1, pp. 7–26, 2003.
- [121] S. Sohrabi, "Customizing the Composition of Web Services and Beyond," Ph.D. dissertation, Depart. of Computer Science, Univ. of Toronto, 2013.
- [122] M. H. Burstein, J. R. Hobbs, O. Lassila, D. Martin, D. V. McDermott, S. A. McIlraith, S. Narayanan, M. Paolucci, T. R. Payne, and K. P. Sycara, "DAML-S: Web Service Description for the Semantic Web," in *Proceedings of the First International Semantic Web Conference on The Semantic Web*, ser. ISWC '02. Springer-Verlag, 2002, pp. 348–363.
- [123] T. Madhusudan and N. Uttamsingh, "A Declarative Approach to Composing Web Services in Dynamic Environments," *Decis. Support Syst.*, vol. 41, no. 2, pp. 325–357, jan 2006.
- [124] U. Kuter and J. Golbeck, "Semantic Web Service Composition in Social Environments," in *Proceedings of the 8th International Semantic Web Conference*, ser. ISWC '09. Springer-Verlag, 2009, pp. 344–358.
- [125] S. Sohrabi and S. A. McIlraith, "Optimizing Web Service Composition While Enforcing Regulations," in *Proceedings of the 8th International Semantic Web Conference*, ser. ISWC '09. Springer-Verlag, 2009, pp. 601–617.
- [126] S. Sohrabi and S. A. McIlraith, "Preference-based Web Service Composition: A Middle Ground Between Execution and Search," in *Proceedings of the 9th International Semantic web Conference on The semantic web - Volume Part I*, ser. ISWC'10. Springer-Verlag, 2010, pp. 713–729.
- [127] "I-X Planning Architecture," Online: accessed Jan. 2014, <http://www.aiai.ed.ac.uk/project/ix/>.
- [128] E. Sirin and B. Parsia, "Planning for Semantic Web Services," in *Semantic Web Services Workshop at 3rd ISWC*, 2004.
- [129] "Pellet: OWL DL Reasoner," Online: accessed Jan. 2014, <http://clarkparsia.com/pellet/>.
- [130] E. Sirin, B. Parsia, and J. Hendler, "Template-Based Composition of Semantic Web Services," in *Proceedings of the AAAI Fall Symposium on Agents and the Semantic Web*. AAAI, 2005, pp. 85–92.
- [131] I. Paik, D. Maruyama, and M. N. Huhns, "A Framework for Intelligent Web Services: Combined HTN and CSP Approach," in *Proceedings of the IEEE International Conference on Web Services*, ser. ICWS'06. IEEE Computer Society, 2006, pp. 959–962.
- [132] N. Lin, U. Kuter, and E. Sirin, "Web service Composition with User Preferences," in *Proceedings of the 5th European Semantic Web Conference on The Semantic Web: Research and Applications*, ser. ESWC'08. Springer-Verlag, 2008, pp. 629–643.
- [133] M. Elkawagy, B. Schattenger, and S. Biundo, "Landmarks in Hierarchical Planning," in *Proceedings of the 2010 Conference on ECAI 2010: 19th European Conference on Artificial Intelligence*. IOS Press, 2010, pp. 229–234.
- [134] M. Elkawagy, P. Bercher, B. Schattenger, and S. Biundo, "Improving Hierarchical Planning Performance by the Use of Landmarks," in *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, 2012, pp. 1763–1769.

- [135] T. A. Estlin, S. A. Chien, and X. Wang, “Hierarchical Task Network and Operator-Based Planning: Two Complementary Approaches to Real-World Planning,” *Journal of Experimental and Theoretical Artificial Intelligence*, vol. 13, no. 4, pp. 379–395, 2001.
- [136] A. Bouguerra and L. Karlsson, “Hierarchical Task Planning under Uncertainty,” in *Proceedings of the 3rd Italian Workshop on Planning and Scheduling*, 2004.
- [137] U. Kuter and D. S. Nau, “Forward-Chaining Planning in Non-Deterministic Domains,” in *Proceedings of the 19th National Conference on Artificial intelligence*, ser. AAAI’04. AAAI, 2004, pp. 513–518.
- [138] M. Brenner and B. Nebel, “Continual Planning and Acting in Dynamic Multiagent Environments,” *Autonomous Agents and Multi-Agent Systems*, vol. 19, no. 3, pp. 297–331, 2009.
- [139] D. D. Corkill, “Hierarchical Planning in a Distributed Environment,” in *Proceedings of the 6th international joint conference on Artificial intelligence - Volume 1*, ser. IJCAI’79. Morgan Kaufmann Publishers Inc., 1979, pp. 168–175.
- [140] M. desJardins and M. Wolverton, “Coordinating a Distributed Planning System,” *AI Magazine*, vol. 20, no. 4, pp. 45–53, 1999.
- [141] J. D. Dix, H. Muñoz Avila, D. S. Nau, and L. Zhang, “IMPACTing SHOP: Putting an AI Planner into a Multi-Agent Environment,” *Annals of Mathematics and AI*, vol. 37, no. 4, pp. 381–407, Apr. 2003.
- [142] F. Amigoni, N. Gatti, C. Pinciroli, and M. Roveri, “What Planner for Ambient Intelligence Applications?” *IEEE Transactions on Systems, Man and Cybernetics, Part A*, vol. 35, no. 1, pp. 7–21, Jan. 2005.
- [143] S. Magnenat, M. Voelke, and F. Mondada, “Planner9, a HTN Planner Distributed on Groups of Miniature Mobile Robots,” in *Proceedings of the 2nd International Conference on Intelligent Robotics and Applications*, ser. ICIRA’09. Springer-Verlag, 2009, pp. 1013–1022.